



华南理工大学
South China University of Technology

本科毕业设计（论文）

基于 FPGA 的高能效脉冲神经网络硬件加速器设计

学 院	微电子学院
专 业	微电子科学与工程
学生姓名	吴宗繁
学生学号	202030050318
指导教师	姚恩义
提交日期	2024 年 6 月 3 日

华南理工大学

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：吴宗繁

日期：2024年6月3日

学位论文授权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保存并向国家有关部门或机构送交论文的复印件和电子版，允许学位论文被查阅；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。本人电子文档的内容和纸质论文的内容相一致。

作者签名：吴宗繁

日期：2024年6月3日

指导教师签名：刘汉

日期：2024年6月3日

作者联系电话：13298249191

电子邮箱：1584221395@qq.com

摘 要

近年来,人工智能开始在越来越多的领域得到应用。然而,随着人工智能性能的不断提高,神经网络层数也随之增加,需要计算的数据量呈指数级增长。一些小型边缘设备难以满足如此高的计算能力和存储需求。为了解决这一问题,人们把目光转向了脉冲神经网络(SNN)。

脉冲神经网络以生物大脑为基础,通过离散的脉冲传递信息,计算速度快,计算功耗低,并且可以实现无监督在线学习。SNN 加速器通过使用硬件平台实现软件算法,具备低成本、低功耗、高并行性等优点。

本文在已有的研究基础上,以低资源开销、低功耗、高计算速度、高通用性为目标,采用流水线、分时复用等电路优化技术,设计了支持在线学习的 SNN 加速器架构。加速器由输入输出模块、存储模块、控制模块、LIF 神经元模块和在线学习模块组成。其中 LIF 神经元模块通过分时复用的方法模拟了 256 个 LIF 神经元和 6 万 4 千个突触。加速器相互之间采用握手协议传递脉冲数据包的方式进行通信。所设计的加速器支持 STDP, SDSP 两种在线学习算法,并采用赢者通吃策略来实现神经元之间的竞争学习。将设计的加速器架构部署到 FPGA 上,综合实现结果表明,LUT 占用 1%,FF 占用 0.55%,存储资源占用 18%,动态功耗为 0.033W,静态功耗为 0.083W。

关键词: 脉冲神经网络; FPGA; 在线学习; STDP; SDSP

Abstract

In recent years, artificial intelligence has begun to be applied in an increasing number of fields. However, with the continuous improvement of AI performance, the number of layers in neural networks has also increased, leading to an exponential growth in the amount of data to be processed. Some small edge devices struggle to meet such high computing and storage demands. To address this issue, attention has turned to Spiking Neural Network (SNN).

Based on the biological brain, SNNs transmit information through discrete spikes, offering fast computation, low power consumption, and the ability to achieve unsupervised online learning. SNN accelerators implement software algorithms on hardware platforms, providing advantages such as low cost, low power consumption, and high parallelism.

Building upon existing research, this study aims to design an SNN accelerator architecture with low resource overhead, low power consumption, high computing speed, and high versatility. Circuit optimization techniques like pipelining and time multiplexing are employed. The accelerator comprises input module, output module, storage module, control module, LIF neuron module and online-learning module. The LIF neuron module simulates 256 LIF neurons and 64,000 synapses using time multiplexing. Communication between accelerators is done via a handshake protocol for transmitting spike data packets. The designed accelerator supports two online learning algorithms, STDP and SDSP, employing a winner-takes-all strategy for competitive learning between neurons. Deploying the accelerator architecture on an FPGA, synthesis results show that LUT usage is 1%, FF usage is 0.55%, storage resource usage is 18%, dynamic power consumption is 0.033W, and static power consumption is 0.083W.

Keywords: Spike Neural Network; FPGA; Online learning; STDP; SDSP

目 录

摘 要	I
Abstract	II
目 录	III
第一章 绪论	1
1.1 研究背景	1
1.2 研究现状	2
1.3 论文结构	3
第二章 脉冲神经网络原理概述	4
2.1 生物神经元介绍	4
2.2 典型的脉冲神经元模型	5
2.2.1 H-H 神经元模型	6
2.2.2 IZH 神经元模型	7
2.2.3 LIF 神经元模型	8
2.3 脉冲神经网络结构	9
2.4 脉冲序列编码介绍	10
2.5 网络学习算法介绍	10
2.5.1 STDP 学习算法	11
2.5.2 SDSP 学习算法	13
2.6 本章小结	14
第三章 在线学习的 LIF 神经元设计	15
3.1 神经元整体结构	15
3.2 神经元各模块的设计与实现	17
3.2.1 神经元更新模块 I	17
3.2.2 神经元更新模块 II	18
3.3 在线学习功能的实现	19

3.4	本章小结	20
第四章	脉冲神经网络加速器设计	21
4.1	加速器整体架构	21
4.2	输入模块	23
4.2.1	脉冲数据包的设置	23
4.2.2	输入数据包的缓存	24
4.3	时间产生模块	24
4.4	存储模块	25
4.5	控制模块	26
4.5.1	控制模块的更新过程	27
4.5.2	神经元的流水线更新	29
4.6	输出模块	29
4.7	本章小结	30
第五章	脉冲神经网络加速器仿真测试	31
5.1	各模块仿真测试	31
5.1.1	LIF 神经元模块仿真测试	31
5.1.2	权重更新模块仿真测试	33
5.1.3	输入模块仿真测试	34
5.1.4	控制模块仿真测试	35
5.1.5	输出模块仿真测试	38
5.2	整体仿真测试	39
5.3	网络仿真测试	44
5.3.1	网络配置	44
5.3.2	网络数据的输入	44
5.3.3	网络训练	45
5.4	资源开销与功耗分析	47
5.5	本章小结	48

结论	49
1. 论文工作总结	49
2. 工作展望	50
参考文献	51
致谢	54

第一章 绪论

1.1 研究背景

人工智能技术已经成为解决各种复杂问题的重要工具之一。传统的基于 CPU 和 GPU 的神经网络加速器在处理大规模神经网络时面临着能耗高和性能瓶颈的挑战。因此,近年来基于 FPGA (Field-Programmable Gate Array) 的神经网络加速器受到了越来越多的关注,其灵活性和并行计算能力使其成为加速神经网络模型运算的理想选择。

人工智能技术历史悠久。从 1943 年至今,研究者们相继提出了许多具有里程碑意义的模型:MP 神经元模型、“感知机”模型、反向传播网络、卷积神经网络、脉冲神经网络、注意力机制等^[1]。

生物大脑具有无与伦比的性能-资源权衡^[2]:蜜蜂大脑有近 100 万个神经元,功耗约为 $10\mu\text{W}$,但它能够进行序列和模式学习、导航、规划和预测等复杂行为,同时表现出比人类婴儿更快的学习速度。传统人工神经网络为了追求更高的计算精确度,导致数据量激增,一些小型边缘计算设备难以满足如此高的计算能力和存储资源的要求。因此,在摩尔定律已经逐渐失效的今天,人们为了使人工智能设备有更接近生物大脑的效率,逐渐把目光转向了具有生物可解释性的脉冲神经网络。

脉冲神经网络受生物大脑处理信息的方式启发,用动力学公式表征神经元节点、用时间离散的脉冲作为神经元之间传递的信号,具有如下优点:

(1) 突触可塑性机制:突触可塑性机制使得脉冲神经网络可以轻松实现无监督在线学习。这种在线学习机制可以为边缘计算设备提供支持。

(2) 动态功耗极低:神经元内部膜电位累积到阈值才会发放脉冲,进而传递信息,发放脉冲后膜电位会恢复到静息电位。在神经网络中大部分神经元膜电位都低于阈值,所以网络中的信息传递具有稀疏性。自适应时钟和事件驱动的方法使得脉冲神经网络的功耗可以进一步降低^[3]。此外,神经元膜电位的累加仅使用加法运算,相比于乘加运算功耗更低。

(3) 抗干扰能力强:输入图像在经过泊松脉冲编码后才会发送给脉冲神经网络。这使得信息具有更高的鲁棒性,不容易受到环境噪声的影响。

1.2 研究现状

脉冲神经网络加速器以其低成本、低功耗、事件驱动的特点，具有广泛的应用潜力，受到了许多国内外知名高校和公司的青睐。下面是一些典型的 SNN 芯片介绍：

(1) TrueNorth^[4] 是 IBM 于 2015 年发布的脉冲神经网络加速器芯片，其采用非冯诺依曼架构，将存储和计算集中在神经形态核心中，以最小化数据移动所消耗的能量。可扩展的二维网格 NoC 连接了芯片上的所有神经形态核心，使 TrueNorth 芯片具有极高的并行性。用户可自定义开发 TrueNorth 芯片的应用，实现语音识别、视觉识别和特征提取等功能。其非冯诺依曼架构和事件驱动架构使 TrueNorth 芯片具有传统处理器无法比拟的低功耗优势：仅使用 63mW 的功耗就能在每秒 30 帧的三通道视频上执行检测和识别 5 个物体的任务。

(2) ODIN^[5] 是由鲁汶大学开发的在线学习数字 SNN 加速器，具有高密度、低功耗和低成本的特点。它采用一阶 LIF 动态神经元模型和支持 20 个 Izhikevich 行为的现象学神经元模型。每个 ODIN 芯片仅含有一个神经形态核心，采用分时复用的方式实现了 256 个神经元。神经元采用同步数字时序电路设计，但其事件驱动的特性使得功耗极低。ODIN 支持 SDSP 在线学习规则，与 STDP 学习规则不同，SDSP 的突触权重更新仅受突触后神经元状态影响，而不考虑脉冲的相对时序，从而实现了低电路成本。不同的 ODIN 芯片可通过 AER 总线连接。在对 MNIST 数据集进行分类时，ODIN 芯片使用单层全连接网络达到了 84.5% 的准确率，功耗仅为 15 nJ。

(3) “天机芯”^[6] 是清华大学开发的一款类脑计算芯片，拥有 156 个 SNN 处理核，超过 4 万个神经元和 1 千万个突触。“天机芯”与吸取了脉冲神经网络与传统人工神经网络的优势，设计了全新的架构，将 ANN 和 SNN 异构融合，使其在加速计算机科学模型与神经科学模型方面均有很好的效果。此外，对于多种混合异构网络，“天机芯”也可以对其进行建模计算。“天机芯”在物体检测、语音识别、自动驾驶等多个人工智能应用场景发挥了强大作用。

1.3 论文结构

本论文旨在设计一种基于 FPGA 的脉冲神经网络加速器。通过将脉冲神经网络的运算从软件层面映射到硬件层面，并对模型与电路结构进行优化设计，提高神经网络计算的效率和性能。论文共分为五章，其余各章主要内容安排如下：

第二章：首先介绍生物神经元的基本概念。接着引出脉冲神经网络中常用的神经元模型：H-H 模型、IZH 模型和 LIF 模型。最后解释脉冲神经网络的结构和工作原理，包括神经元之间的连接方式、脉冲编码方式和在线学习算法。

第三章：详细介绍了 LIF 神经元模块与在线学习模块的设计过程，并对脉冲神经元模型进行优化，使其在满足精度需求的条件下消耗较小的资源。

第四章：描述整个加速器的结构框架，详细介绍了支持在线学习机制的 SNN 加速器整体设计过程，包括输入输出模块，控制模块，存储模块等，并阐述各模块的功能和行为特性。

第五章：实验仿真环节，首先对上述设计的模块分别进行仿真测试，验证其功能的正确性。接着对 SNN 加速器整体进行仿真测试。最后使用 MNIST 数据集对网络进行训练，分析对比训练前后权重的变化，并对综合实现后的 SNN 加速器整体进行资源统计和功耗分析。

在论文的最后，对本次工作进行了总结，并给出了未来的工作展望。

第二章 脉冲神经网络原理概述

神经元是生物神经系统的基本组成单位。当神经元接收到足够的电化学刺激时，其膜电位会发生变化，达到一定的阈值时会产生动作电位，从而传播信号到与其连接的其他神经元。为了适应不同计算需求，研究人员提出了多种脉冲神经元模型^[7]。

动作电位是生物神经系统中信息传递的基本单位，其时序和频率等特性对于信息处理至关重要。因此，研究人员提出了多种网络拓扑结构和脉冲编码方式。

生物体如何进行学习和记忆一直以来是研究的热门，脉冲神经网络以生物神经元为基础，在无监督在线学习方面有巨大优势。

本章将首先介绍生物神经元模型，并以此为基础引出常见的脉冲神经元模型，最后对脉冲神经网络结构、编码方式和学习算法进行介绍。

2.1 生物神经元介绍

神经元细胞和神经胶质细胞构成了生物的神经系统。神经元细胞是神经系统中的基本功能单位。神经元细胞参与生物大脑的信息处理和传递过程，生物大脑复杂的结构和多样的功能得益于神经元细胞的不同类型和连接模式。神经胶质细胞为神经元提供支持和保护^[8]。

神经元主要由细胞体（Soma）、树突（Dendrite）、轴突（Axon）和突触（Synapse）组成，它们在神经元产生和传递信息时发挥着各自的作用。当神经元细胞受到足够的刺激时，其细胞膜内部电位会大于阈值，从而产生一个不衰减的神经冲动^[9]。神经冲动沿着轴突迅速传播。轴突是神经元细胞上的细长突起，它具有特殊的电生理特性，能够有效地传导神经冲动。轴突会与下一个神经元细胞的树突或细胞体连接，实现信息传递的目的，连接部位称为突触。当神经冲动传播到突触时，会有一个电信号到化学信号再到电信号的转变。接收到神经冲动的突触前膜，会释放神经递质到突触间隙中。神经递质是一种包含信息的化学物质，它通过扩散作用跨过突触间隙，与突触后膜上的受体结合。这些受体位于下一个的神经元的树突或细胞体上。神经递质与受体结合后，会转变为电信号，使下一个神经元的细胞膜电位发生变化。如果这种变化足够大，使下一个神经元的膜电位也超过了阈值，则可以产生新的神经冲动，继续向下传播。上述的信息传递与

处理的过程是生物神经网络的基础。在大脑中，数以亿计的神经元相互连接，信息源源不断地被这些神经元接收、处理、发送。生物神经元结构和神经元之间的连接如图 2-1 所示。

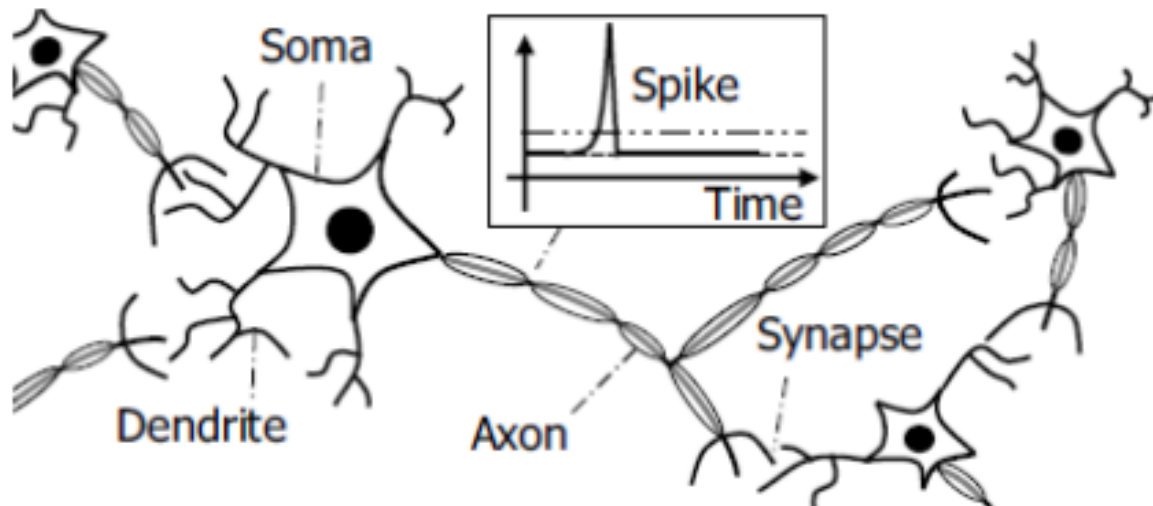


图 2-1 生物神经元结构和神经元之间的连接^[8]

2.2 典型的脉冲神经元模型

为适应不同常见的计算需求，研究人员提出了多种脉冲神经元模型。它们在生物可解释性的程度上有所不同，一般来说，生物可解释性越高，所需要的计算量就越大，模型越难以适用于大规模脉冲神经网络中。常见脉冲神经元模型主要有以下三类：

(1) 具有生物可解释的生理模型，如 H-H (Hodgkin-Huxley) 模型^[10]。这类模型基于对神经元细胞膜离子通道的详细生物物理学了解，能够模拟神经元的生理行为，例如动作电位的生成和传播。

(2) 脉冲生成机制的非线性模型，如 IZH (Izhikevich) 模型^[11]。这类模型舍弃了一部分生物可解释性，换来了更简单的数学形式。IZH 模型在可以较好描述神经元产生脉冲的动态特性的同时，拥有良好的计算效率。

(3) 固定阈值的脉冲发放模型，如 LIF (Leaky Integrate and Fire) 模型^[12]。这类模型的生物可解释性较差，但计算效率很高。模型假设神经元细胞膜电位达到固定阈值时产生脉冲，忽略了复杂的生物物理机制，在一些大规模计算任务中具有强大的优势。

接下来，我们将对这些典型模型和它们的计算特性进行介绍。

2.2.1 H-H 神经元模型

Hodgkin 和 Huxley 基于实验观察的离子通道行为和离子流动的物理原理，提出了 H-H(Hodgkin-Huxley)神经元模型。该模型描述了神经元膜电位如何随时间变化以及如何受到外部刺激的影响，不仅对各种神经轴突具有普遍意义，也揭示了各种可兴奋性细胞的基本规律。图 2-2 为 H-H 神经元模型等效电路图。

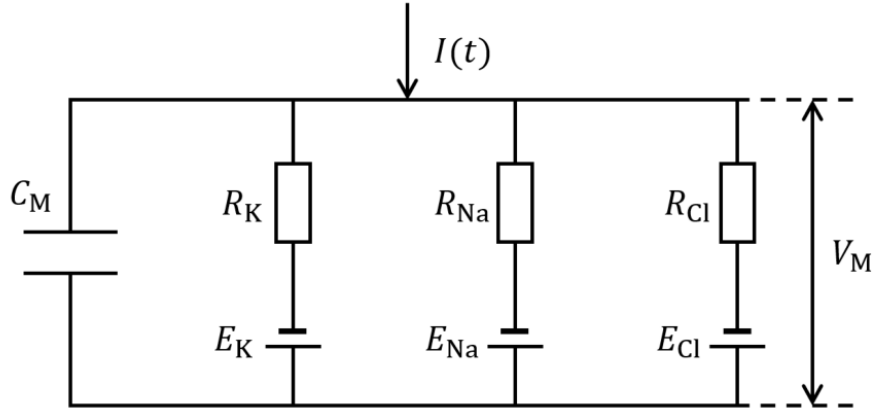


图 2-2 H-H 神经元模型等效电路图

钠离子通道、钾离子通道和泄漏通道是 H-H 模型的重要组成部分。这些通道控制离子进出细胞膜，从而导致神经元膜电位的变化。在模型中，每个通道的状态都可以由一组微分方程描述，这些方程描述了通道的开放和关闭状态。H-H 模型的主要方程如下：

$$\begin{cases} C_m \frac{dV}{dt} = -g_L(V - E_L) - \bar{g}_{Na} m^3 h (V - E_{Na}) - \bar{g}_K n^4 (V - E_K) + I \\ \frac{dm}{dt} = \alpha_m(V)(1 - m) - \beta_m(V)m \\ \frac{dh}{dt} = \alpha_h(V)(1 - h) - \beta_h(V)h \\ \frac{dn}{dt} = \alpha_n(V)(1 - n) - \beta_n(V)n \end{cases} \quad (2-1)$$

式中， C_m 表示膜电容； g_L 表示漏电导； E_L 表示静息电位； $\bar{g}_{Na}$ 和 $\bar{g}_K$ 分别表示钠离子与钾离子通道的最大电导值； E_{Na} 和 E_K 分别表示对应的逆转电位； m ， h ， n 表示细胞膜上的活化门。图 2-3 展示了 H-H 神经元模型膜电位变化与各个活化门之间的关系。

尽管 H-H 模型提供了对神经元行为更精细的描述，可以模拟动作电位的产生、传播和终止等现象，但在实际应用中，由于模型的高维非线性方程计算量大，难以对大规模脉冲神经网络进行模拟，同时模型参数众多，难以找到适合具体问题的参数。

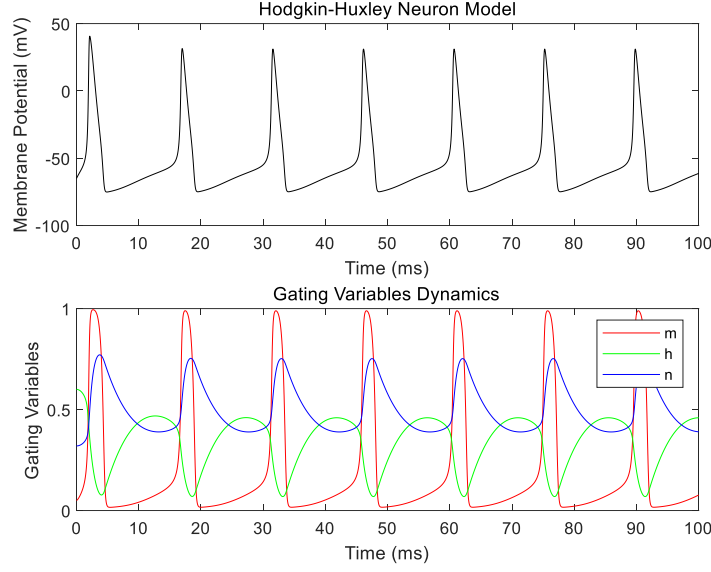


图 2-3 H-H 神经元模型仿真图

2.2.2 IZH 神经元模型

Izhikevich 在 2003 年提出了 IZH (Izhikevich) 神经元模型。模型可以通过复位机制表达丰富的神经元特性，很好地模拟了神经元的兴奋和抑制行为，但是模型没有离子通道机理，其参数也不具生物学意义。IZH 模型由两个微分方程组成，分别描述了膜电位的变化和膜电位恢复变量的变化：

$$\begin{cases} \frac{dV}{dt} = 0.04V^2 + 5V + 140 - U + I \\ \frac{dU}{dt} = a(bV - U) \end{cases} \quad (2-2)$$

如果神经元的膜电位大于阈值，辅助的复位机制为：

$$\begin{cases} V \leftarrow c \\ U \leftarrow U + d \end{cases} \quad (2-3)$$

其中， V 表示膜电位； U 表示恢复变量； a 、 b 、 c 、 d 均为无量纲参数。IZH 模型通过调整参数 a 、 b 、 c 、 d 和外部输入电流 I ，可以模拟出不同类型神经元的兴奋性和抑制性行为。

图 2-4 分别展示了大脑皮层神经元（上图） $a = 0.02$ ， $b = 0.2$ ， $c = -65$ ， $d = 8$ 和猫的皮质神经元（下图） $a = 0.02$ ， $b = 0.2$ ， $c = -50$ ， $d = 2$ 的仿真图。参数设置的多样性使得该模型在仿真和理解神经网络行为时具有很大的灵活性和实用性。如果使用 CORDIC 算法对 IZH 神经元模型进行优化，可以获得更高的计算效率^[13]。

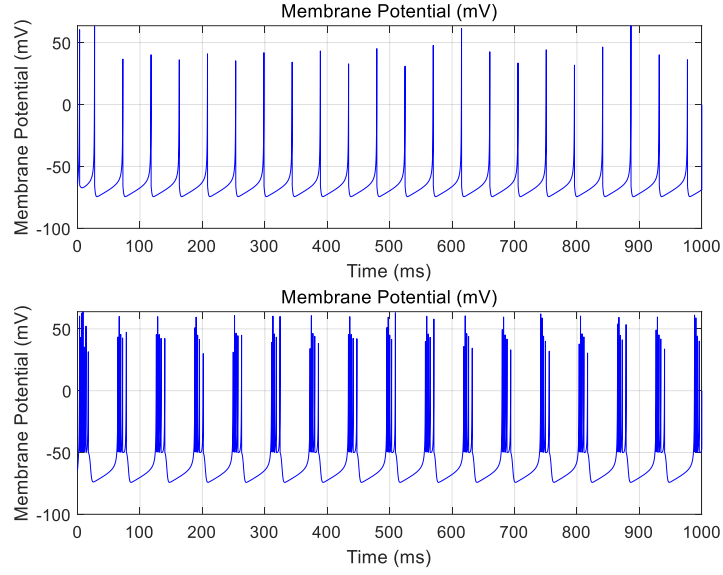
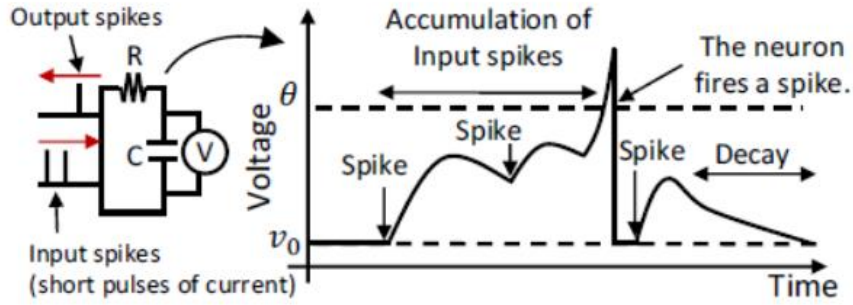


图 2-4 不同参数的 IZH 神经元模型仿真图

2.2.3 LIF 神经元模型

LIF (Leaky Integrate-and-Fire) 神经元模型是一种简化的神经元模型，常用于建模神经网络中的单个神经元。LIF 模型更多是用在大规模脉冲神经网络的计算上，虽然模型会忽略神经元的一些复杂行为，但它仍然是研究神经网络基本性质和行为的有用工具。


 图 2-5 LIF 模型等效电路与特性图^[11]

LIF 神经元模型的主要特征和方程如下：

$$\begin{cases} \tau_m \frac{dV}{dt} = -(V - V_{rest}) + R_m I \\ V = V_{rest}, V \geq V_{th} \end{cases} \quad (2-4)$$

其中 V 是膜电位， τ_m 是膜时间常数， V_{rest} 是静息膜电位， V_{th} 是神经元阈值， R_m 是膜电阻， I 是输入电流。如果神经元膜电位达到阈值，则会被重置为静息电位 V_{rest} ，同时发放脉冲。如果设置了不应期，则神经元膜电位会在这段时间内一直保持为静息电位（图 2-6）。此外，LIF 模型在没有外部输入时膜电位 V 会向静息膜电位 V_{rest} 恢复。

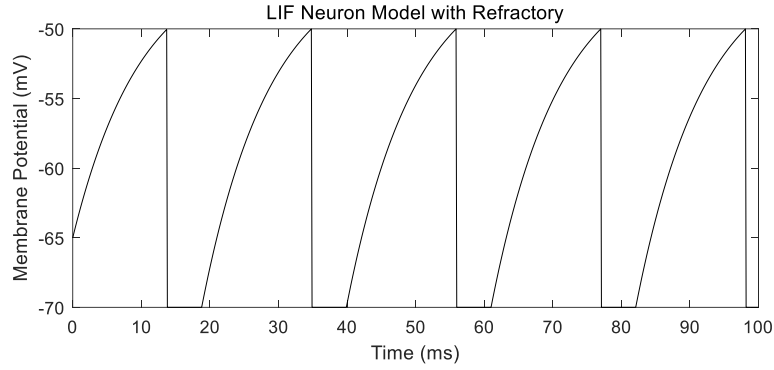


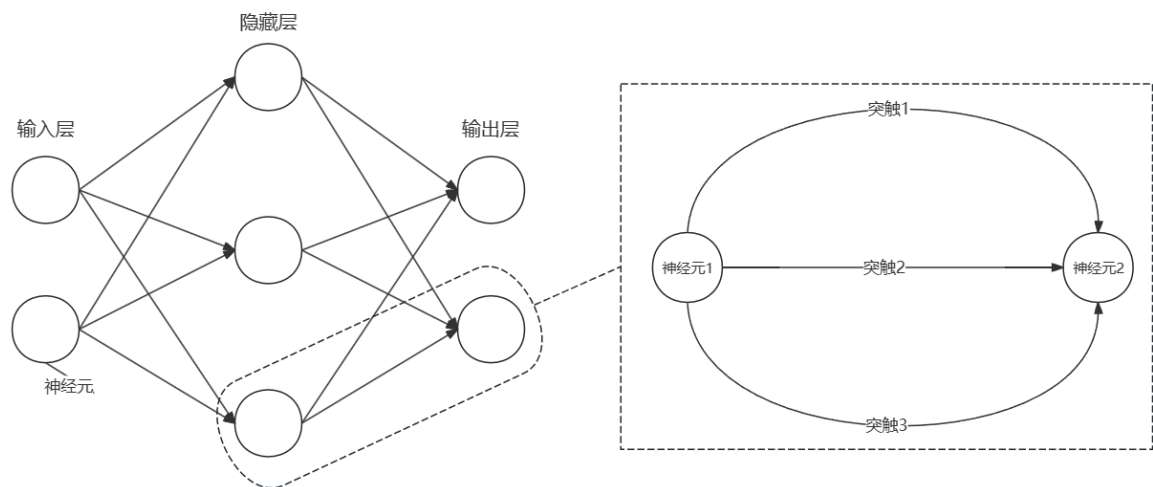
图 2-6 带有不应期的 LIF 神经元模型仿真图

2.3 脉冲神经网络结构

脉冲神经网络具有多种拓扑结构，如：前馈网络、递归网络和卷积网络^[14]等，其中前馈网络是最简单且经典的网络结构，广泛用于各种脉冲神经网络的研究。

前馈网络由输入层、隐藏层和输出层组成，其中输入层和输出层为单层，而隐藏层可以为多层。信息在网络中只能前向传播，即从输入层传播到隐藏层，最后传播到输出层。前馈网络一般被用于神经网络最后的全连接层。

传统神经网络的全连接层为神经元一对一连接，两个神经元之间仅有一个权重数据。得益于脉冲的时间特性，脉冲神经网络可以实现两个神经元之间具有多突触连接的结构。如图 2-7 所示。脉冲的频率、延迟都可以作为信息的一部分，对于两个神经元之间的多突触连接结构，每个突触连接可以有不同的突触权重和脉冲延迟，这样使突触前神经元与突触后神经元之间的作用关系更为紧密。

图 2-7 多层前馈脉冲神经网络结构与多突触连接^[15]

2.4 脉冲序列编码介绍

脉冲神经网络的输入和内部信息的传递都是一系列的脉冲，被称为脉冲序列。单个脉冲是信号传递的基本单元。研究人员在观察生物神经元发放的脉冲后发现，生物神经元中的脉冲波形基本相似，所以认为信息与脉冲波形无关。虽然现有研究还无法确定生物神经网络究竟使用怎样的脉冲编码方式，但通过脉冲的发放频率来编码的频率编码和通过脉冲的发放时间来编码的时间编码是现在脉冲神经网络研究中常用的两种编码方式。

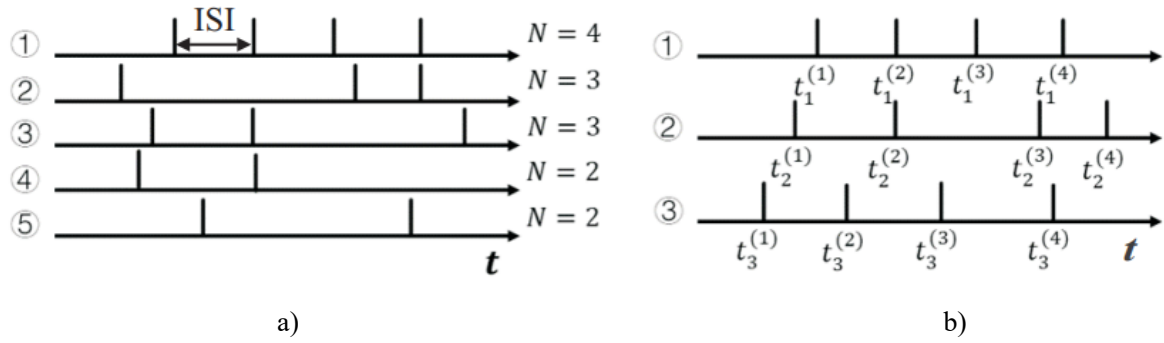


图 2-8 编码方式示意图^[15]。(a) 频率编码，(b) 时间编码

在频率编码中，刺激的强度对应脉冲的频率，强度越大、频率越高。在时间编码中，信息通过脉冲的时序关系进行编码。任何脉冲序列都有其自身的时间发放模式，因此与刺激有关的信息可以由脉冲的精确发放时间来表达。时间编码可以用更少的脉冲来表示与频率编码相同的信息，从而有效提高神经网络的计算速度。

2.5 网络学习算法介绍

脉冲神经网络的学习算法分为有监督算法和无监督算法。有监督算法如 **SpikeProp**^[16] 和 **Tempotron**^[17] 依赖梯度下降和反向传播，通过迭代运算调整网络参数以提高准确度。然而，这种方法面临梯度消失、速度缓慢和运算量大等挑战。在硬件化阶段，这些算法也面临着计算不友好的问题，因此通常用于离线学习，即在计算机上训练网络，然后将参数加载到硬件中。

无监督学习算法可用于处理海量无标签数据，避免了人工添加标签的巨大成本。常用的无监督学习算法采用赢者通吃策略 (**Winner-Takes-All, WTA**)，即识别层的神经元相互竞争，每时刻只有一个神经元被激活，其他则被抑制。这种算法通常以 **Hebb** 规则为

基础，不需要梯度下降和反向传播，因此实现简单，适用于在线学习。

下面我们将介绍 STDP (Spike Timing Dependent Plasticity) 和 SDSP (Spike-driven Synaptic Plasticity) 两种无监督学习算法。

2.5.1 STDP 学习算法

Hebbian 学习规则^[18]是一种最早的突触可塑性规则，该规则简单而强大，奠定了神经科学中突触可塑性研究的基础。Hebbian 学习规则认为当神经元 A 频繁地激活并引起神经元 B 的激活时，突触连接从 A 到 B 的强度会增强。这一规则被简洁地概括为：神经元之间的连接强度取决于它们共同活动的频率和时序。

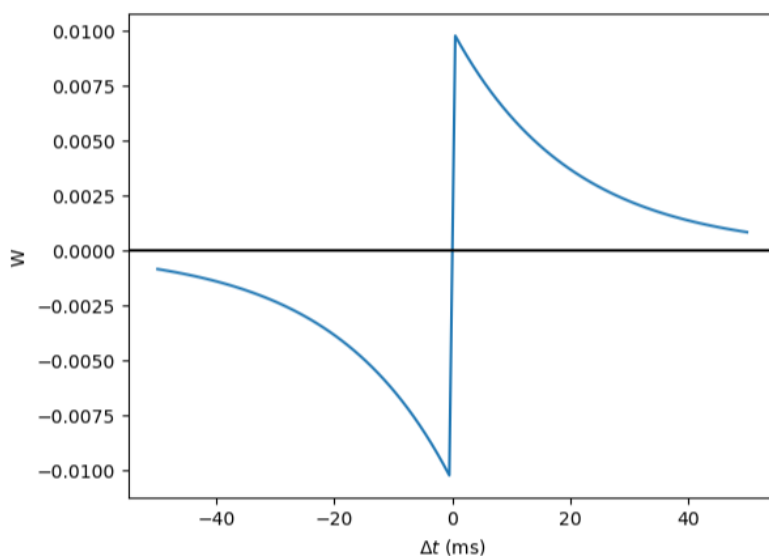


图 2-9 经典 STDP 算法曲线图

在此基础上，神经科学家 Gerstner 等人于 1996 年研究发现：神经元之间活动模式会影响它们的突触连接强度：如果突触后神经元是在接收到了的突触前神经元传来的脉冲信号后才发放的脉冲，则它们之间的突触连接增强，如果突触后神经元在发放了脉冲之后才接收到突触前神经元发送来的脉冲信号，则它们之间的突触连接减弱，进而提出了 STDP 学习算法^[19]。

STDP 算法将这种时序相关性建模为一种学习规则，如公式 (2-5) 所示。这样，神经元对于它们共同参与的活动模式可以形成偏好，从而加强相关的信息传递。STDP 算法的形式主要有两种，一种是基于时间窗的，在一定的时间窗内，对两个神经元的活动时间差进行统计，从而调整连接的强度；另一种是连续的，基于实际的活动时间差进行

实时的连接强度调整。

然而，直接使用计算机模拟公式（2-5）进行 STDP 学习的效率非常低，因为我们必须记录时间窗内的所有神经元脉冲。为了加快仿真速度，通常使用带有突触痕迹（synaptic traces）的 STDP 算法^[20]。

$$\Delta w(\Delta t) = \begin{cases} A_p \exp\left(\frac{-\Delta t}{\tau_p}\right), & \Delta t > 0 \\ -A_d \exp\left(\frac{\Delta t}{\tau_p}\right), & \Delta t < 0 \end{cases} \quad (2-5)$$

其中， A_p 和 A_d 分别为权重增加和减小的参数， τ_p 为时间常数。在此算法中，除了突触权重外，每个突触还记录了突触前迹 x_{pre} 的值，突触前迹用来模拟突触前脉冲近期发放的次数。每当突触前脉冲到达突触时， x_{pre} 就会增加 1，否则 x_{pre} 会以指数形式递减。当突触后脉冲到达突触时，权重变化量 Δw 根据突触前迹计算得出，如公式（2-6）所示：

$$\Delta w = \eta(x_{pre} - x_{tar})(w_{max} - w)^\mu \quad (2-6)$$

其中 η 是学习率， w_{max} 是权重的最大值， μ 决定了权重变化量对变化前权重的依赖性。 x_{tar} 是突触后脉冲发放时突触前迹的目标值。突触权重的变化量取决于突触前迹的值与目标值之差。当突触前迹的值固定时，目标值设置的越大，权重的变化量就越小，甚至为负。这样可以使得当突触前神经元发放很少时，突触连接增加缓慢或是减弱。由于突触前迹对时间具有指数依赖性，而权重变化量与突触前迹的值成正比，所以上述 STDP 学习规则对时间具有指数依赖，相比于固定权重变化量的学习规则，更加合乎生物原理。

在上述使用突触前迹的 STDP 算法的基础上，通过加入突触后迹，可以得到基于对称学习规则的 STDP 算法。突触后迹的工作方式与突触前迹相同，当突触后脉冲出现时，突触后迹的值加一，否则会指数形式递减。此外，突触前脉冲和突触后脉冲的出现都会使连接权重更新。当突触前脉冲出现时，权重变化量由公式（2-7）计算：

$$\Delta w = -\eta_{pre}x_{post}w^\mu \quad (2-7)$$

其中 η_{pre} 是突触前脉冲的学习率。突触后脉冲出现时的权重变化量计算方式与公式（2-6）相同。

随着研究的逐渐推进，更多的 STDP 算法被提出：如使用循环网络结构的“Recurrent STDP”^[21]，使用储备池网络结构的“Reservoir STDP”^[22]和加入反向传播算法的“BP-STDP”^[23]等。

2.5.2 SDSP 学习算法

Brader 等人进而通过引入钙变量来模拟突触的记忆特性，提出了 SDSP 学习算法^[24]。SDSP 算法中引入的钙变量 $C(t)$ 是突触后脉冲活动的函数，由公式 (2-8) 描述。

$$\frac{dC(t)}{dt} = -\frac{1}{\tau_c} C(t) + J_c \sum_i \delta(t - t_i) \quad (2-8)$$

其中， τ_c 是一个较大的时间常数， J_c 是单个突触后脉冲对钙变量变化的贡献，求和项为在时间 t_i 到达的突触后脉冲的总和。

图 2-10 展示了 SDSP 学习算法中突触权重 W 的变化。公式 (2-9) 是对算法的具体描述。其中 a 、 b 是突触权重的改变量， t_{pre} 是突触前神经元脉冲到达的时间， V_{mem} 是神经元的膜电位， θ_m 、 θ_1 、 θ_2 、 θ_3 都是参数值。如果在当前时间没有突触前脉冲输入或是神经元的参数值不满足公式 (2-9) 的条件，则会触发突触的双稳态效应，使得突触权重朝着 W_{max} 或是 W_{min} 的方向改变。相比于 STDP 算法，SDSP 算法没有指数计算，更适合电路实现^[25]。

$$\begin{cases} W \rightarrow W + a & \text{if } V_{mem}(t_{pre}) \geq \theta_m \text{ and } \theta_1 \leq C(t_{pre}) \leq \theta_3 \\ W \rightarrow W - b & \text{if } V_{mem}(t_{pre}) < \theta_m \text{ and } \theta_1 \leq C(t_{pre}) \leq \theta_2 \end{cases} \quad (2-9)$$

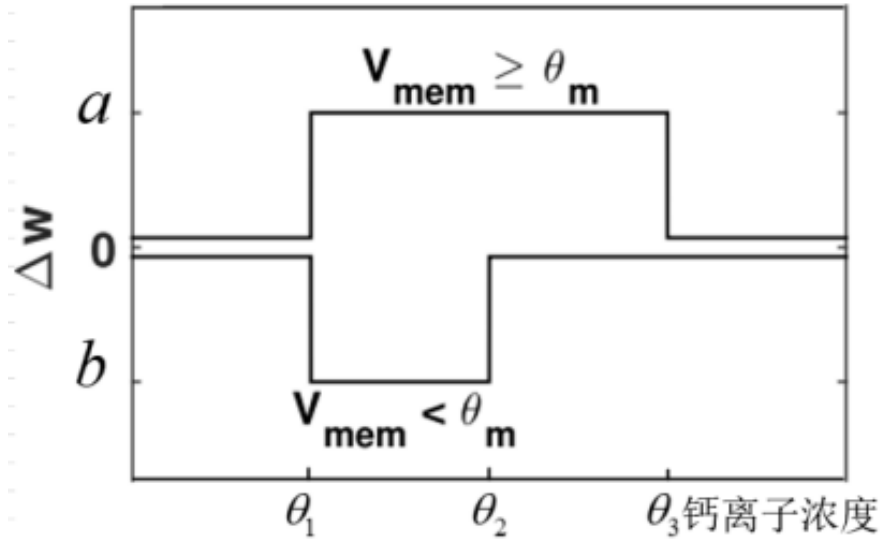


图 2-10 SDSP 学习规则

2.6 本章小结

本章首先介绍了生物神经元的基本结构与行为特性，由此引出了三种代表性的脉冲神经元模型，其中对于生物特性模拟效果最好的是 H-H 神经元模型，对于大规模脉冲神经网络运算效率最高的是 LIF 神经元模型，IZH 神经元模型介于它们之间，具有较好的生物特性与计算效率。接着介绍了脉冲神经网络的网络结构，其中最简单与常见的是前馈型脉冲神经网络。频率编码、时间编码是两种常见的脉冲编码方式。最后，介绍了不同的网络学习算法，并对当下热门且易于实现的两种无监督学习算法：STDP 和 SDSP 学习算法，进行了阐述说明。

第三章 在线学习的 LIF 神经元设计

3.1 神经元整体结构

LIF 神经元顶层模块包括了更新模块 I，更新模块 II，如图 3-1 所示。LIF 神经元模块用于计算由存储模块发来的神经元数据和突触数据。表 3-1 为 LIF 神经元模块端口信号说明，其中部分信号由多个信号组成，具体如表 3-2 所示。

LIF 神经元模块内部的所有数据均为定点数。规定膜电位和权重的低七位为小数位，膜电位为无符号数，由 16bit 二进制数表示，对应脉冲神经网络中的膜电位取值范围： $0 \leq w < 512$ 。权重为有符号数，由 8bit 二进制数表示，对应脉冲神经网络中的权重取值范围为： $-1 \leq w < 1$ 。

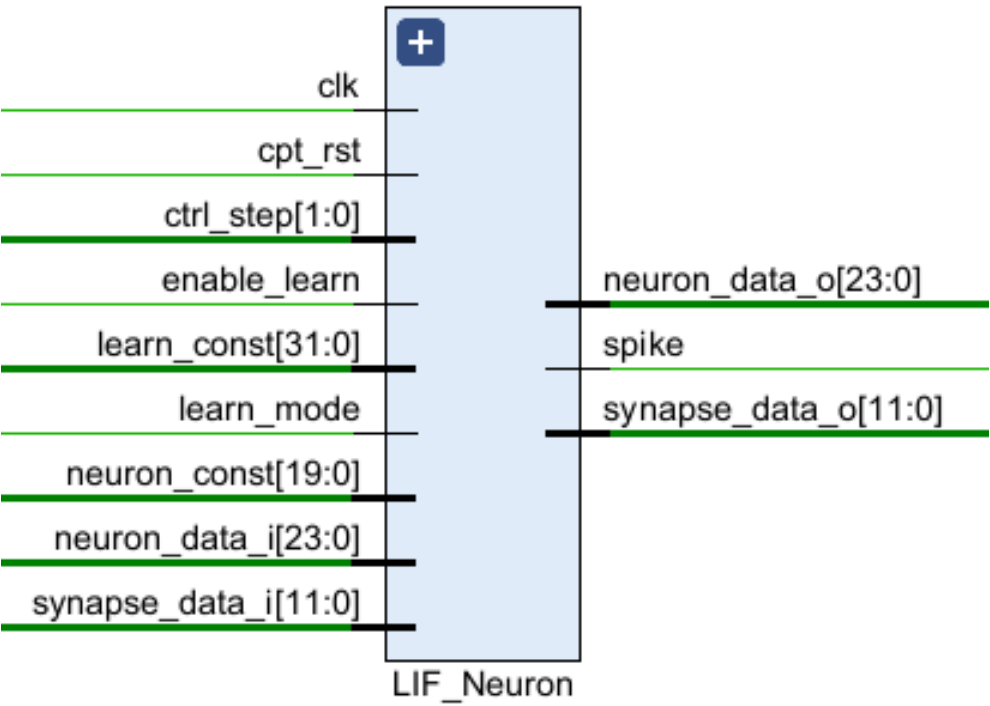


图 3-1 LIF 神经元整体结构图，左边为输入信号，右边为输出信号

表 3-1 LIF 神经元模块内部所有信号说明

序号	信号名称	位宽(bit)	类别	描述
1	<i>clk</i>	1	输入	系统时钟
2	<i>ctrl_step</i>	2	输入	LIF 神经元模块工作模式
3	<i>cpt_rst</i>	1	输入	竞争学习重置标志
4	<i>learn_mode</i>	1	输入	学习模式选择
5	<i>neuron_const</i>	20	输入	神经元参数（常数）
6	<i>learn_const</i>	32	输入	学习算法参数（常数）
7	<i>neuron_data_i</i>	24	输入	输入的神经元数据
8	<i>synapse_data_i</i>	12	输入	输入的突触权重数据
9	<i>neuron_data_o</i>	24	输出	输出的神经元数据
10	<i>synapse_data_o</i>	12	输出	输出的突触权重数据
11	<i>spike</i>	1	输出	是否发放脉冲

表 3-2 数据的组成

神经元数据: <i>neuron_data_i(o)</i>			
位置	信号名称	位宽 (bit)	描述
15-0	<i>v_mem_i(o)</i>	16	神经元的膜电位
19-16	<i>t_ref_i(o)</i>	4	神经元的不应期时间
23-20	<i>calcium_i(o)</i>	4	神经元的钙变量
突触数据: <i>synapse_data_i(o)</i>			
7-0	<i>weight_i(o)</i>	8	突触权重
11-8	<i>learn_var_i(o)</i>	4	学习变量
神经元参数 (常数)			
15-0	<i>v_thr</i>	16	设置的阈值电位
19-16	<i>t_ref</i>	4	设置的不应期时间
学习算法参数 (常数)			
3-0	<i>xtar</i>	4	STDP 参数 $xtar$
19-4	<i>theta_m</i>	16	SDSP 参数 θ_m
23-20	<i>ca_theta_1</i>	4	SDSP 参数 θ_1
27-24	<i>ca_theta_2</i>	4	SDSP 参数 θ_2
31-28	<i>ca_theta_3</i>	4	SDSP 参数 θ_3

3.2 神经元各模块的设计与实现

3.2.1 神经元更新模块 I

LIF 神经元的更新模块 I (Neuron_Update_I) 如图 3-2 所示，为组合逻辑电路。该模块会在每个时间步的最开始工作，对每个已经激活的目标神经元更新膜电位、钙离子和不应期，为接下来的神经元更新 II 做准备。

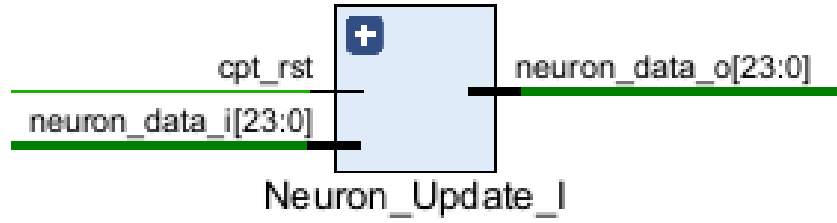


图 3-2 LIF 神经元更新模块 I，左边为输入信号，右边为输出信号

LIF 神经元模型和钙变量的变化方程均由微分方程描述（公式（2-4）、公式（2-9）），无法直接用数字电路实现。对此我们可以使用欧拉方法，将微分方程变为差分的形式，转换后的差分方程如公式（3-1）和（3-2）所示。

$$V[t] = V[t - 1] + \frac{dt}{\tau_v} (V_{rest} - V[t - 1] + I[t]) \quad (3-1)$$

$$Ca[t] = Ca[t - 1] - \frac{dt}{\tau_c} Ca[t - 1] + J_c \sum_i \delta(t - t_i) dt \quad (3-2)$$

除法操作在数字电路中是一项十分消耗资源与时间的运算，所以我们选用右移运算来代替除法运算，同时对于右移后的数据，小数部分大于 5 的进位，小于等于 5 的舍去。设置神经元膜电位的时间常数为 32，即右移 5 个单位，设置钙变量的时间常数为 8，即右移 3 个单位。

如果信号 t_ref_i 大于零，则说明神经元正处在不应期内，更新模块 I 会在将该信号的值减 1，在 t_ref_o 端口输出。

当在线学习启用时，如果上一时间步有目标神经元发放， cpt_rst 信号会被置为高电平，更新模块 I 会重置所有输入的目标神经元的膜电位和不应期，以达到竞争学习的目的。

3.2.2 神经元更新模块 II

LIF 神经元的神经元更新模块 II (Neuron_UPDATE_II) 如图 3-3 所示, 为组合逻辑电路。该模块除了具有更新神经元膜电位, 判断是否发放脉冲的功能之外, 还具有为 STDP 算法更新突触前迹和为 SDSP 算法标识突触变化方向的功能。

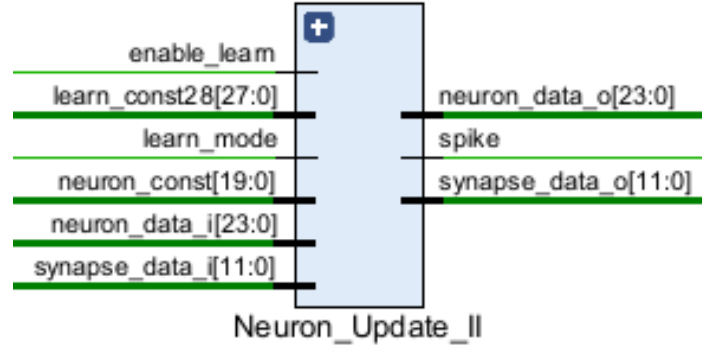


图 3-3 LIF 神经元更新模块 II, 左边为输入信号, 右边为输出信号

神经元膜电位的变化受到输入电流 $I[t]$ 的影响, 而 $I[t]$ 是 t 时刻输入脉冲所引起的电流变化。对于目标神经元 j , 有 n 个源神经元与其具有突触连接, 突触权重为 $w_{i,j}$ 。突触权重的正负代表突触为兴奋性突触或是抑制性突触, 进而在有脉冲输入时, 控制目标神经元的膜电位会增加或减少 $w_{i,j}$ 。公式 (3-3) 描述了突触权重 $w_{i,j}$ 与输入电流 $I[t]$ 的关系。

$$I[t] = \sum_{i=1}^n w_{i,j} \delta(t - t_i) \quad (3-3)$$

其中, $\delta(t - t_i)$ 为 t_i 时刻输入的突触前脉冲。当目标神经元处于不应期外时, 神经元的膜电位会累加输入电流的值。如果更新后的神经元膜电位大于阈值, 则会触发复位机制, 将膜电位置为静息电位, 并拉高脉冲发放标志信号 $spike$ 。此时, 神经元进入设置好的不应期, 而在不应期内, 神经元的膜电位会一直保持在静息电位。

当启用 STDP 在线学习时, 神经元的钙变量会一直保持为零。突触的学习变量用于表示 STDP 算法中的 x_{pre} 参数, 更新模块 II 会将输入的 x_{pre} 值加一, 放在输出端口。

当启用 SDSP 在线学习时, 如果有目标神经元发放脉冲, 则其钙变量会增加 2。突触的学习变量用于表示突触变化的方向, 更新模块 II 会根据公式 (2-9), 满足正向变化时该参数会被设置为 4'b0001, 负向变化设置为 4'b0010, 除此之外 4'b0000 和 4'b0011 均为双稳态变化。权重更新模块会根据参数的值与学习算法对突触权重进行更新。

3.3 在线学习功能的实现

在线学习功能被设计在每个时间步的所有目标神经元的膜电位更新完成后进行。这是因为使用 STDP 学习算法更新权重时，需要知道所有突触前迹的值；使用 SDSP 学习算法更新权重时，需要知道权重是否需要进行双稳态变化。在线学习功能由权重更新模块（Weight_Update）实现，如图 3-4 所示，模块根据学习变量的值对发放脉冲的目标神经元权重进行更新。

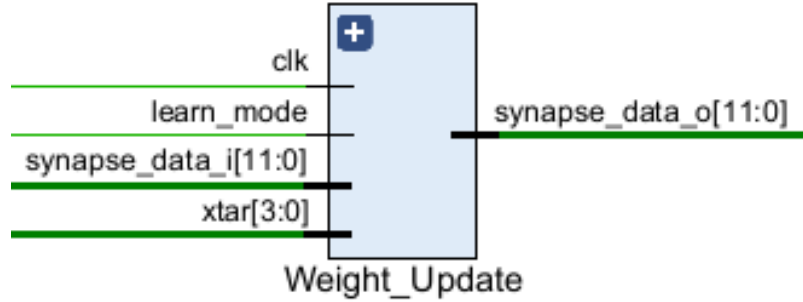


图 3-4 LIF 神经元权重更新模块，左边为输入信号，右边为输出信号

当启用 STDP 学习时，学习变量用于表示算法中的 x_{pre} 。在第二章中所描述的基于突触痕迹的 STDP 学习算法中，权重变化量的计算公式包含乘法和指数，这对于电路实现来说会消耗许多资源，所以我们在设计电路时取 $\mu = 0$ ，并且由于规定了 LIF 神经元内部信号的低 7 位为小数位，所以取 $\eta = 1/128$ ，这样可以省去对 η 乘法电路的设计，以精度的略微下降换取电路资源的更少消耗，这样公式（2-6）就简化为：

$$\Delta w = x_{pre} - x_{tar} \quad (3-4)$$

此外，在 Δw 计算完成之后，还会对 x_{pre} 进行衰减操作，基于 x_{pre} 的位宽，设置其时间常数为 8，与膜电位的衰减一样，电路使用右移代替除法，同时小数部分大于 5 则会进位，小于等于 5 则会被舍去。

当启用 SDSP 学习时，学习变量表示权重更新的方向，模块会根据参数的不同使权重增加或减小。如果输入权重为权重的最大值或最小值，则不会产生变化。

3.4 本章小结

本章介绍了 SNN 加速器中最核心的 LIF 神经元模块和权重更新模块的设计，通过使用欧拉方法，适当缩放，用移位代替除法等优化方法，极大减小了 LIF 神经元模块和权重更新模块的资源消耗。这两个模块通过模拟生物神经元的行为特性，实现了脉冲发放，突触学习等功能，是脉冲神经网络推理和学习的基础。

第四章 脉冲神经网络加速器设计

4.1 加速器整体架构

SNN 加速器由输入模块接收上一级的脉冲数据，通过控制模块控制数据的流向，利用 LIF 神经元模块进行状态更新和突触权重学习，最后从输出模块输出脉冲数据到下一级。在加速器中，除了上述提到的模块之外，还包括神经元模块、突触存储模块、时间产生模块和权重更新模块这些模块组合起来，构成了完整的系统。图 4-1 为加速器整体架构图。

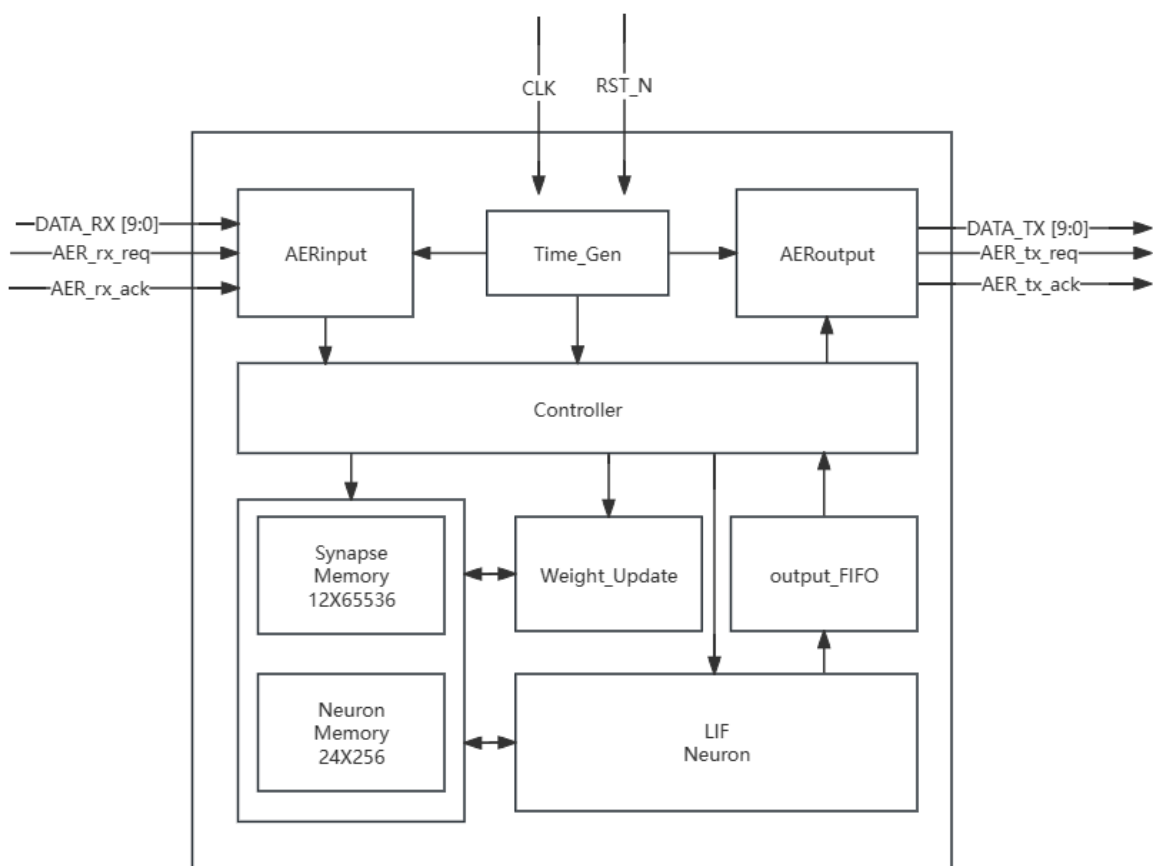


图 4-1 SNN 加速器整体架构图

SNN 加速器的各个模块概述如下：

- (1) 输入模块：通过握手协议接收输入的脉冲数据包，根据脉冲数据包中的时间标识存入对于的输入缓存 FIFO 中，在对应的时间步，发送给控制模块。
- (2) 时间步产生模块：利用系统时钟来产生时间步。

- (3) 存储模块: 包括位宽 24bit, 深度为 256 的神经元内存和位宽 12bit, 深度 65536 的突触权重内存。
- (4) 控制模块: 具有多种工作状态, 不同状态下控制神经元模块对数据进行不同类型的更新。此外, 还有控制存储模块读取与写入的功能。
- (5) LIF 神经元模块: 模拟生物神经元中膜电位衰减、累积, 脉冲发放等功能。
- (6) 权重更新模块: 采用 STDP 学习算法或是 SDSP 学习算法, 更新突触权重。
- (7) 输出模块: 编码生成脉冲数据包, 通过握手协议向外部发送。

SNN 加速器中所用到的所有信号的说明如表 4-1 所示, 表 3-1 中已经解释过的内容在表 4-1 中不再出现。

表 4-1 SNN 加速器中的信号说明

序号	信号名称	位宽(bit)	描述
1	<i>rst_n</i>	1	系统复位
2	<i>enable_learn</i>	1	是否启用在线学习
3	<i>interval</i>	20	时间步长度
4	<i>timestep0</i>	1	时间步的第 0 位
5	<i>src_number</i>	8	激活的源神经元数量
6	<i>tar_number</i>	8	激活的目标神经元数量
7	<i>ctrl_step</i>	2	控制模块状态
8	<i>inputFIFO_pop</i>	1	输入 FIFO 弹出标志
9	<i>inputFIFO_empty</i>	1	输入 FIFO 空标志
10	<i>inputFIFO_full</i>	1	输入 FIFO 满标志
11	<i>outputFIFO_pop</i>	1	输出 FIFO 弹出标志
12	<i>outputFIFO_empty</i>	1	输出 FIFO 空标志
13	<i>outputFIFO_full</i>	1	输出 FIFO 满标志
14	<i>source_addr</i>	8	源神经元地址
15	<i>target_addr</i>	8	目标神经元地址
16	<i>neuron_addr_R</i>	8	读神经元内存地址
17	<i>neuron_W</i>	1	写神经元内存使能
18	<i>neuron_addr_W</i>	8	写神经元内存地址
19	<i>synapse_addr_R</i>	16	读突触权重内存地址
20	<i>synapse_W</i>	1	写突触权重内存使能
21	<i>synapse_addr_W</i>	16	写突触权重内存地址

4.2 输入模块

输入模块（AERinput）用于接收外部发来的脉冲数据包并保存到标识的输入缓存 FIFO 中。图 4-2 为输入模块端口图。

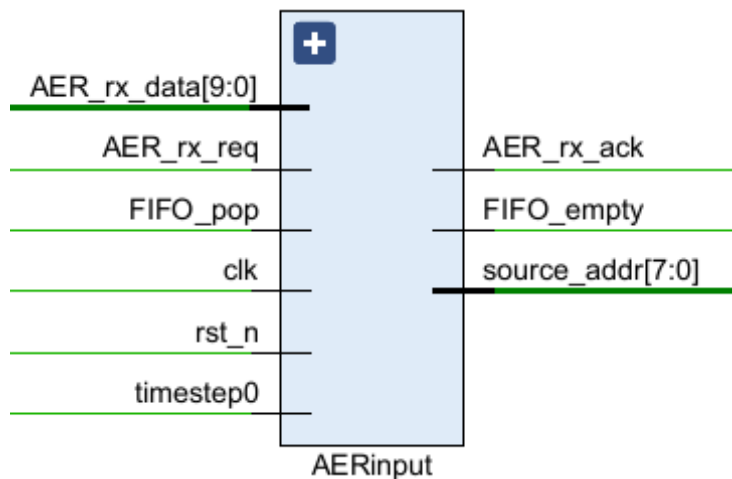


图 4-2 输入模块端口图，左边为输入信号，右边为输出信号

4.2.1 脉冲数据包的设置

脉冲神经网络由于其信号的稀疏性，具有低功耗的优势，所以 SNN 加速器常用地址事件表示（Address Event Representation, AER）的方法进行网络间的数据交互。为了简化网络中的数据传输量，我们设计的 AER 数据包的位宽为 10 位，如表 4-3 所示。

表 4-3 加速器输入脉冲数据包说明

数据包中的位置	位宽	描述
7-0	0bit	发放脉冲的源神经元编号
8	1bit	源神经元发放脉冲的时间步
9	1bit	数据包有效标识位，1 有效

AER 数据包采用异步握手协议的方式进行通信。因为 SNN 加速器的设计基于同步电路，而 AER 数据包是通过异步的方式输入的，所以我们选用异步握手协议来实现数据包的发送与接收。脉冲数据包的异步握手协议如图 4-3 所示。

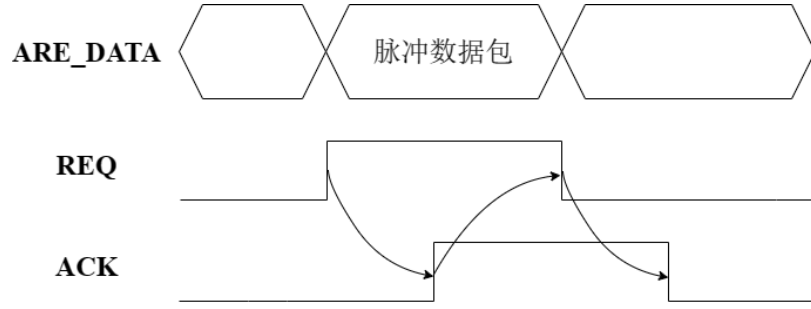


图 4-3 异步握手协议图

4.2.2 输入数据包的缓存

SNN 加速器在处理本时间步的数据时，还需要将下一个时间步要处理的数据缓存起来，因此需要 2 个 FIFO 来对数据进行缓存分别记为 0 号 FIFO 和 1 号 FIFO。数据包的最低位是源神经元发放脉冲的时间步，解码模块会根据此信息，将 0 时刻发放的源神经元脉冲存入 1 号输入缓存 FIFO，并在 1 时刻发送给控制模块和目标神经元；将 1 时刻的源神经元脉冲存入 0 号输入缓存 FIFO，并在 0 时刻发送给控制模块和目标神经元。两个数据缓存 FIFO 的容量均设计为 $256 \times 8\text{bit}$ ，以满足输入的需求。

4.3 时间产生模块

时间产生模块用于产生时间步信号，图 4-4 为模块端口图。时间产生模块的主要功能相对简单，使用一个计数器来产生时间步信号。如何确定时间步的长度是我们需要关心的问题。

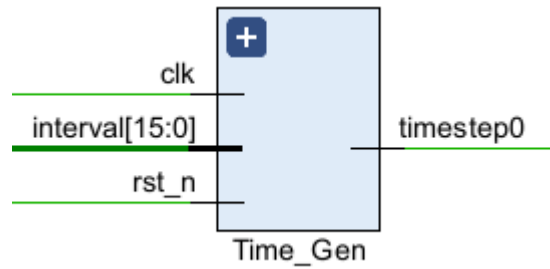


图 4-4 时间产生模块端口图，左边为输入信号，右边为输出信号

对于 SNN 加速器，当有一个源神经元脉冲输入时，控制模块会对所有启用的目标神经元进行有序更新。在控制模块的 UPDATE II 状态下，假设有 n 个目标神经元与源神经元有突触连接，通过使用流水线设计，需要 $n + 2$ 个系统时钟周期便可以完成所有启用的目标神经元的更新。

通常，一个时间步中往往有多个源神经元脉冲输入，并且除了 UPDATE II 更新阶段，还存在 UPDATE I 和 LEARN 更新阶段。UPDATE I 更新阶段的时间可以由启用的目标神经元数量直接得到，但是 LEARN 更新阶段的时间与在本时间步内发放脉冲的神经元数量有关。假设有 m 个源神经元和 n 个目标神经元，且在 n 个目标神经元中有 a 个神经元发放脉冲，则最小时间步可以由公式（4-1）计算。

$$t_n = (m + 1)(n + 2) + a(m + 2) \quad (4-1)$$

4.4 存储模块

存储模块分为神经元存储模块和突触权重存储模块两部分，如图 4-5 所示。

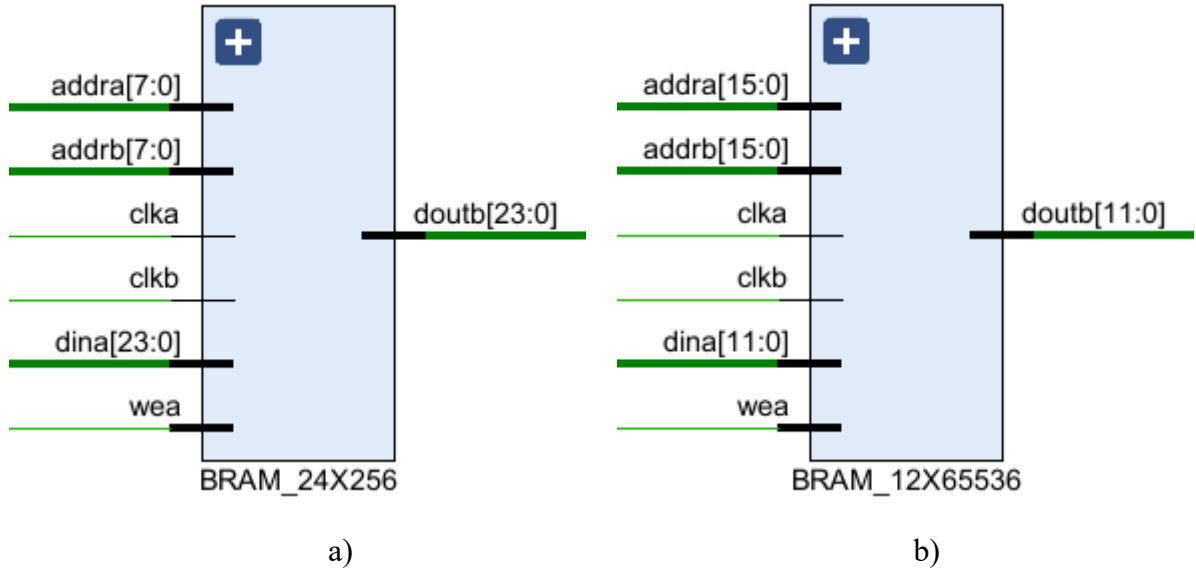


图 4-5 存储模块端口图，左边为输入信号，右边为输出信号。

（a）神经元存储，（b）突触权重存储

神经元存储模块用于存储神经元的膜电位、不应期信息和钙离子变量，采用位宽为 24bit、深度为 256 的双端口 RAM。内存中存储的神经元位置与编号一一对应，即编号为 i 的神经元存储在第 i 行。突触权重存储模块用于存储 12bit 的突触权重，采用位宽为 12bit、深度为 65536 的双端口 RAM。表 3-2 说明了 24bit 的神经元数据格式和 12bit 的突触数据格式。第 i 个源神经元和第 j 个目标神经元之间的突触连接地址如公式（4-2）所示：

$$synapse_{addr} = \{neuron_{addr i}, neuron_{addr j}\} \quad (4-2)$$

存储模块消耗了 SNN 加速器的大量资源。为了节约资源，一些如神经元阈值、SDSP

学习参数等常数值被保存在同一个寄存器中，这种设计会使每个目标神经元不能单独对这些参数进行设置，位于同一个 SNN 加速器上的目标神经元将会共享这些参数，而对于不同 SNN 加速器上的目标神经元，这些值可以进行分别设置。

4.5 控制模块

控制模块是 SNN 加速器的核心模块，用于控制 SNN 加速器中的数据流动。图 4-6 为控制模块的端口图。

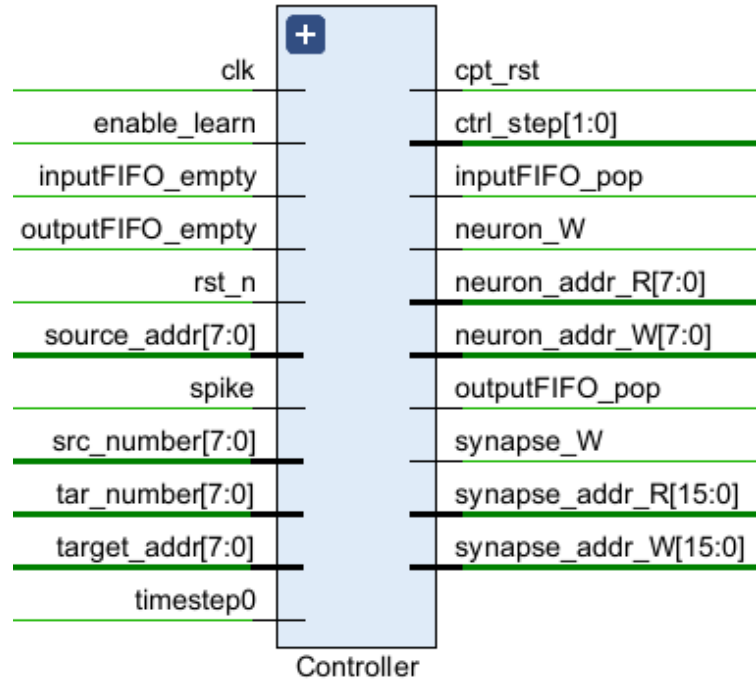


图 4-6 控制模块端口图，左边为输入信号，右边为输出信号

控制模块的状态转移图，如图 4-7 所示。当系统开始信号有效或新的时间步开始时，控制模块状态由 IDLE 转为 UPDATE I，使用 LIF 神经元中的更新模块 I 对所有激活的目标神经元进行泄漏更新与不应期更新等操作；然后状态由 UPDATE I 转为 UPDATE II，控制模块会根据设定好的步骤，将需要读取与写入的地址发送给存储模块，将更新模式控制信号发送给 LIF 神经元模块；当输入缓存 FIFO 为空时，如果没有启用在线学习，则说明本时间步的更新结束，控制模块状态由 UPDATE II 转为 IDLE；如果启用在线学习且输出缓存 FIFO 非空时，则控制模块状态由 UPDATE II 转为 LEARN，控制 LIF 神经元模块进行突触权重更新；当输出缓存 FIFO 为空时说明权重更新完成，控制模块状态由 LEARN 转为 IDLE。

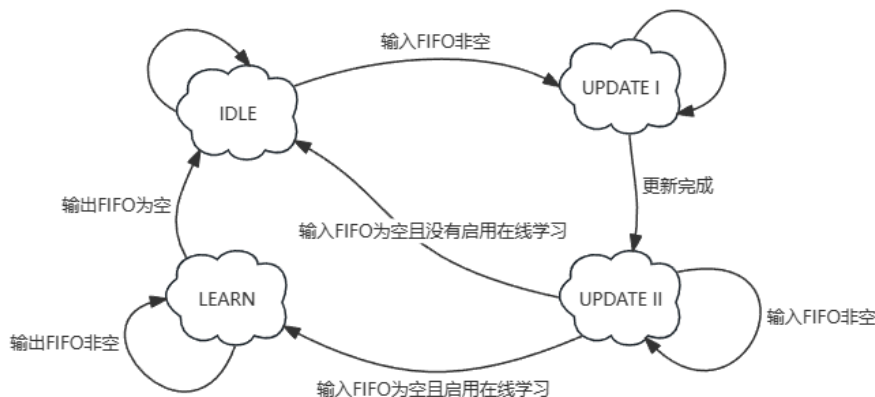


图 4-7 控制模块状态转移图

4.5.1 控制模块的更新过程

控制模块处在 UPDATE I 状态时，会从存储模块中依次取出目标神经元的神经元数据对其进行更新，并在下一个时钟周期写回存储模块。当 cpt_rst 信号为高电平时，回写的数据将被设置为 0，表示在竞争中获胜的神经元对其余神经元的侧抑制作用。

控制模块处在 UPDATE II 状态时，会根据输入源神经元脉冲依次对 N 个目标神经元进行更新。执行该操作的流程图如图 4-8 所示，具体步骤如下：

- (1) 提取发放脉冲的源神经元：根据当前时间步，从输入缓存 FIFO 中读取源神经元地址 i 。初始化目标神经元地址 $j = 0$ 。
- (2) 神经元与突触信息读取：根据读取的源神经元地址 i 和目标神经元地址 j 通过公式 (4-2) 计算得到突触地址，将目标神经元地址和突触地址送入存储模块。
- (3) 神经元与突触信息更新：从存储模块读出的信息将被直接送入 LIF 神经元模块进行计算。如果目标神经元发放脉冲，则其地址会被存入输出缓存 FIFO。
- (4) 更新结果回写：经 LIF 神经元模块更新后的神经元数据和突触数据会被回写到存储模块中。如果启用了在线学习则神经元数据和突触数据均会被回写，反则则仅会回写神经元数据。
- (5) 切换到下一个目标神经元：对目标神经元地址递增： $j = j + 1$ ，并判断 j 与 N 的关系。如果此时 $j = N$ 说明已经对所有激活的目标神经元完成更新，控制模块会通知对应的输入缓存 FIFO 弹出数据。如果 $j < N$ 则会跳转到步骤 (1) 进行下一个目标神经元的更新。

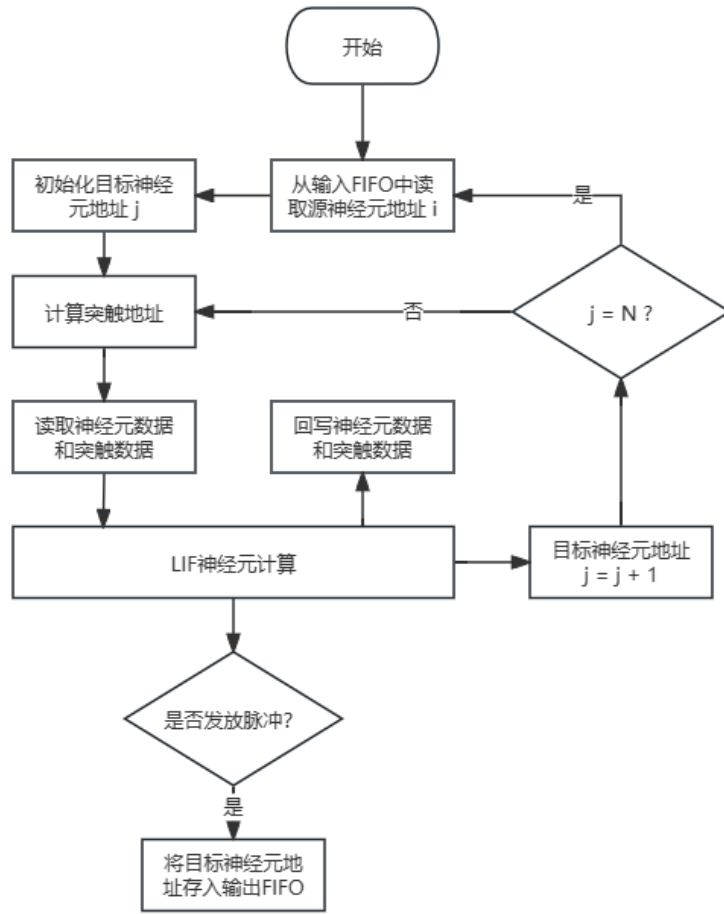


图 4-8 控制模块 UPDATE II 状态流程图

控制模块处在 LEARN 状态时，会根据在本时间步内发放脉冲的目标神经元地址，对其对应的突触权重进行更新，假设有 M 个源神经元被激活，具体步骤如下：

- (1) 提取发放脉冲的目标神经元：从输出缓存 FIFO 中读取目标神经元地址 j 。初始化源神经元地址 $i = 0$ 。
- (2) 突触信息读取：根据读取的源神经元地址 i 和目标神经元地址 j 通过公式 (4-2) 计算得到突触权重地址，将地址送入存储模块。
- (3) 突触信息更新：在 LEARN 状态，LIF 神经元模块仅会从存储模块的突触存储中读取突触数据，根据其中的学习变量与设定的学习算法更新突触权重。
- (4) 更新结果回写：将更新后的突触信息回写到突触存储模块中。
- (5) 切换到下一个突触：对源神经元地址递增： $i = i + 1$ ，并判断 i 与 M 的关系。如果此时 $i = M$ 说明所有突触更新完毕，控制模块会通知对应的输出缓存 FIFO 弹出数据。如果 $i < M$ 则会跳转到步骤 (1) 进行下一个突触的更新。

4.5.2 神经元的流水线更新

在 SNN 加速器对目标神经元的膜电位的更新过程中，采用了流水线设计，以求最大限度利用所例化的一个物理神经元。下面是将一个物理神经元通过流水线的方法分时复用为 256 个目标神经元的例子。在时钟周期 t 读取目标神经元 t 的状态信息和对应的源-目突触权重信息。在时钟周期 $t + 1$ 的时候更新目标神经元 t 的状态，同时将神经元 $t + 1$ 的状态信息和对应的源-目突触权重信息从存储模块中取出。在时钟周期 $t + 2$ 的时候回写目标神经元 t 的状态，更新目标神经元 $t + 1$ 的状态，同时将神经元 $t + 2$ 的状态信息和对应的源-目突触权重信息从存储模块中取出。使用流水线的方式将原本需要 512 个时钟周期的更新时间缩短为 258 个时钟周期，大大加快了 SNN 加速器的识别与学习速度。由于需要在一个时钟周期内对存储模块进行读和写，因此在设计时需要选用简单双端口的 BRAM。

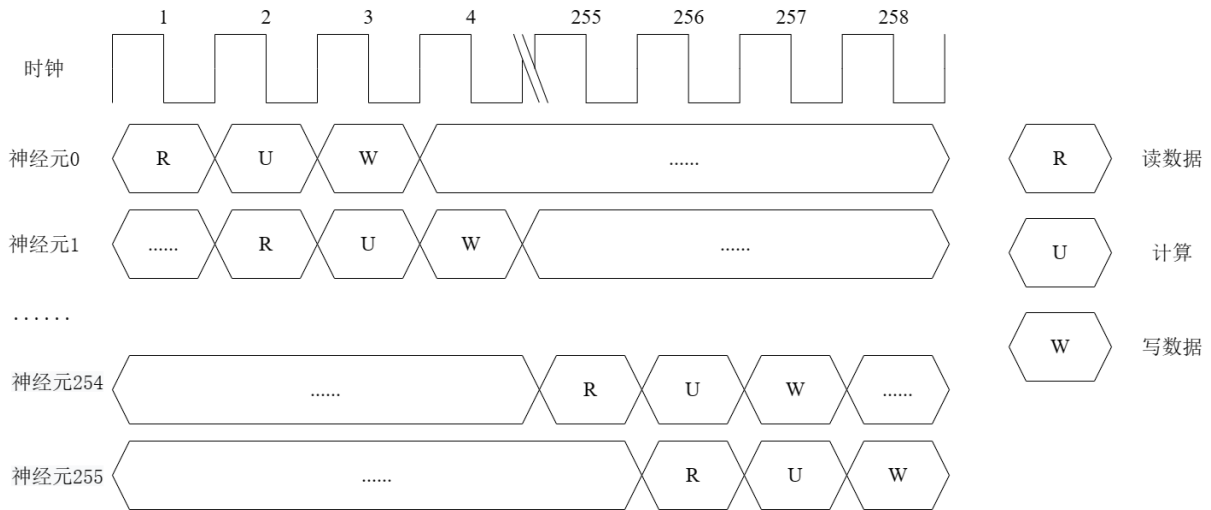


图 4-9 流水线更新时序图

4.6 输出模块

输出模块会根据发放脉冲的目标神经元地址和当前时间步编码生成脉冲数据包，发送给接收端。输出端生成的脉冲数据包格式与输入模块相同。输出模块如图 4-10 所示。

输出模块同样采用异步握手协议的方式传输 AER 数据。通过握手协议传递一次数据的时间较长，而在一个时间步内可能会连续有多个目标神经元进行脉冲发放，这时就需要一个输出缓存 FIFO 来将达到发放的目标神经元信息缓存起来。输出缓存 FIFO 的大小设置为 $256 \times 8\text{bit}$ 。与输入模块不同的是，输出缓存 FIFO 并没有位于输出模块内部，

这是因为在线学习启用时，LIF 神经元模块也需要从输出缓存 FIFO 中取出达到发放的目标神经元地址，进行突触权重的更新。

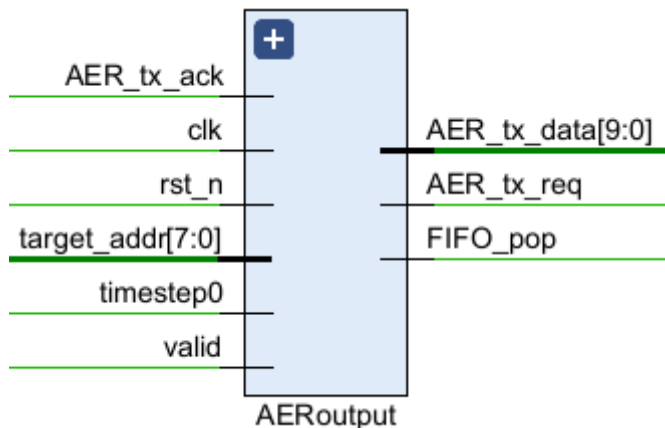


图 4-10 输出模块端口图，左边为输入信号，右边为输出信号

4.7 本章小结

本章介绍了 SNN 加速器的架构和其余模块的设计。输入模块和输出模块用来使不同的 SNN 加速器之间可以互相交流，传递信息；控制模块类似于 SNN 加速器的“大脑”，指挥加速器的工作流程；存储模块保存了神经元和突触的信息；时间产生模块用于控制 SNN 加速器中每一个时间步的长短。

第五章 脉冲神经网络加速器仿真测试

在本章中我们首先会对 SNN 加速器各模块进行单独的仿真测试，以验证各模块功能的正确性。在确定各模块功能均正确后，对 SNN 加速器整体进行仿真测试，验证整体功能的正确性。最后我们通过搭建一个前馈型脉冲神经网络，选用第二章介绍的带有突触前迹的 STDP 在线学习算法和 MNIST 数据集对网络进行训练，验证 SNN 加速器的理论功能的正确性。最后，分析综合实现后 SNN 加速器的资源开销与功耗。

5.1 各模块仿真测试

5.1.1 LIF 神经元模块仿真测试

本节对 LIF 神经元模块进行仿真验证与结果分析。为了便于仿真和观察，设置目标神经元个数为 1，且仅有 1 个突触与之相连，该突触的权重在每个时钟周期随机更新，同时为了使神经元的发放更为明显，该突触权重有 90% 的概率为正（兴奋性突触），10% 的概率为负（抑制性突触）。设置时间步为 16 个时钟周期，其中在第 1 个时钟周期内 LIF 神经元进行 UPDATE I 的更新，其余时钟周期内 LIF 神经元进行 UPDATE II 更新。其余常数设置如表 5-1 所示。

表 5-1 LIF 神经元仿真常数设置

符号	描述	值
v_{thr}	神经元阈值电位	16'd768
t_{ref}	神经元不应期时间	4'd1
θ_m	SDSP 参数 θ_m	16'd256
θ_1	SDSP 参数 θ_1	4'd3
θ_2	SDSP 参数 θ_2	4'd8
θ_3	SDSP 参数 θ_3	4'd13
τ	STDP 参数	4'd8

图 5-1 所示为神经元膜电位趋势图。由于兴奋性突触的概率较大，在脉冲的持续输入下，膜电位整体呈现上升趋势。当膜电位到达阈值时，神经元会进行发放， $spike$ 信号拉高一个时钟周期。同时当神经元发放后的一个时间步内，其膜电位将一直保持为静息

电位。

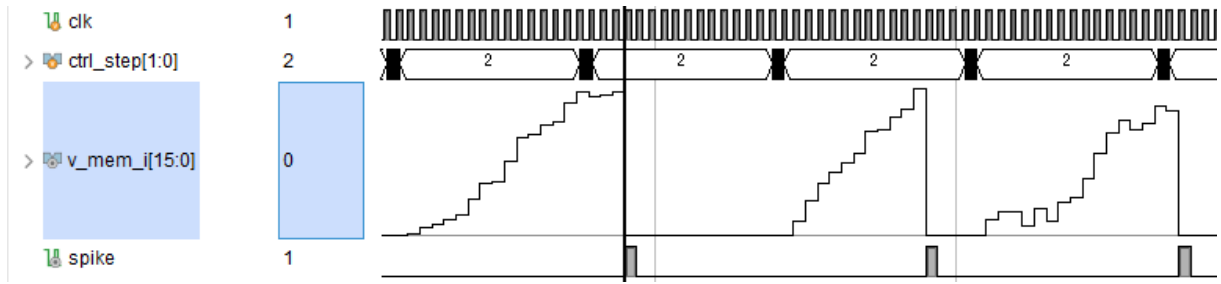


图 5-1 神经元膜电位趋势图

图 5-2 验证了膜电位累加的正确性。输入膜电位为 0，突触权重为 12，输出膜电位为 12；接着输入膜电位为 12，突触权重为 69，输出膜电位为 81，以此类推。可以看出膜电位正确累加。

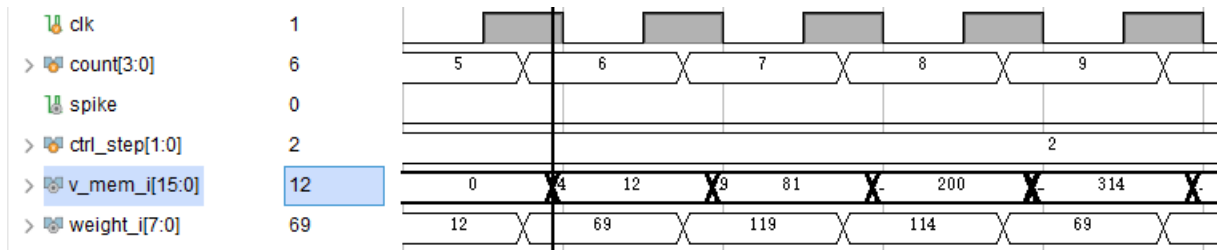


图 5-2 神经元膜电位累加图

图 5-3 展示了神经元达到阈值后各个信号的变化，当神经元到达阈值后，膜电位重置为静息电位，脉冲发放信号 $spike$ 拉高，不应期标志信号 t_ref_i 被置为 1，钙变量增加 2 个单位。

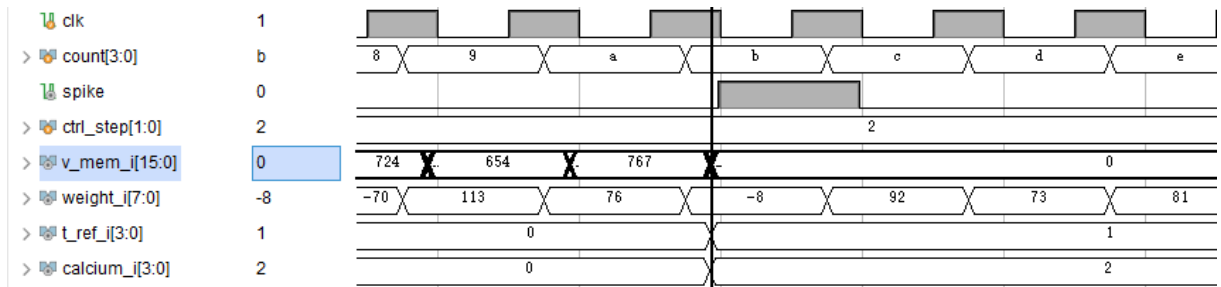


图 5-3 神经元发放后各信号变化

图 5-4 展示了 UPDATE I 阶段对膜电位和钙变量的衰减操作，用于模拟生物神经元中电荷的泄漏。图中输入膜电位为 744，衰减结果为 $744 \times 31/32 = 720.75 \approx 721$ 。输入膜电位为 7，衰减结果为 $7 \times 7/8 = 6.125 \approx 6$ 。

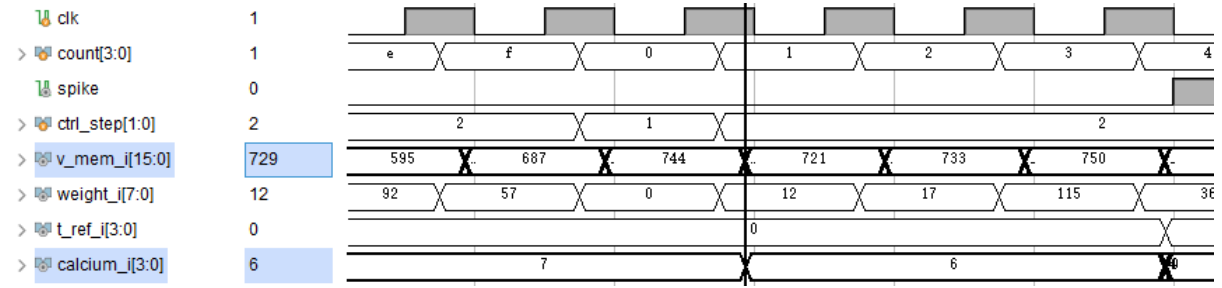


图 5-4 UPDATE I 阶段计算结果图

图 5-5 和图 5-6 分布展示了启用 SDSP 学习和 STDP 学习时学习变量的变化。在图 5-5 中的标记时刻前，膜电位和钙变量满足公式 (2-9) 中使突触权重下降的条件，学习变量被赋值为 4'h2，而在标记时候之后，膜电位和钙变量满足公式 (2-9) 中使突触权重上升的条件，学习变量被赋值为 4'h1。在图 5-6 中，可以看到只要有脉冲输入，对应突触的学习变量就会加一。



图 5-5 启用 SDSP 学习时学习变量的变化

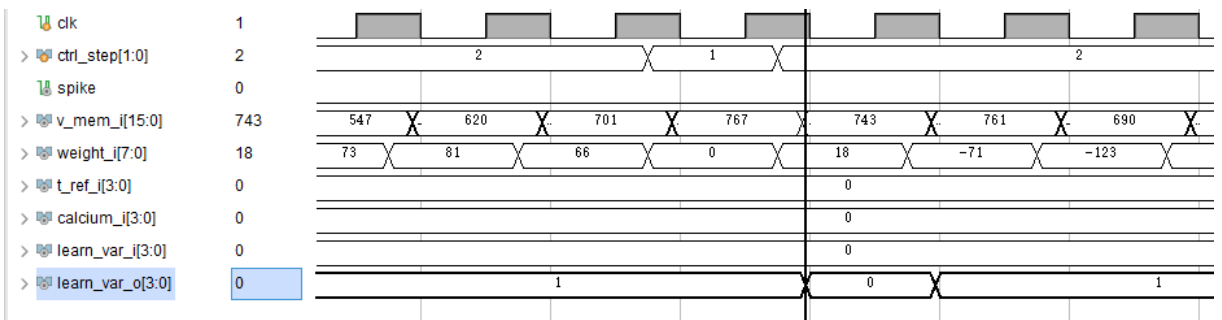


图 5-6 启用 STDP 学习时学习变量的变化

5.1.2 权重更新模块仿真测试

图 5-7 为启用 STDP 在线学习时权重更新模块的信号波形图。根据公式 (3-4)，权重变化量等于学习变量与参数 x_{tar} 之差，可以发现在光标处，输入权重为 -122，学习变量为 0，所以按照规则，输出权重为 $-122 + (0 - 8) = -130$ ，但是由于权重被限制在了

-128 到 127 之间，所以输出权重的真实值为-128。对于光标前后的数据，其运算结果经检验也是正确的： $-76 + (10 - 8) = -74$ ， $-14 + (4 - 8) = -18$ 。此外，权重更新模块还会对输入的学习参数进行衰减操作：当输入学习变量为 13 时，输出学习变量为 $13 \times 7/8 = 11.375 \approx 11$ ，当输入学习变量为 5 时，输出学习变量为 $5 \times 7/8 = 4.375 \approx 4$ ，输入学习变量为 0 则不会衰减。

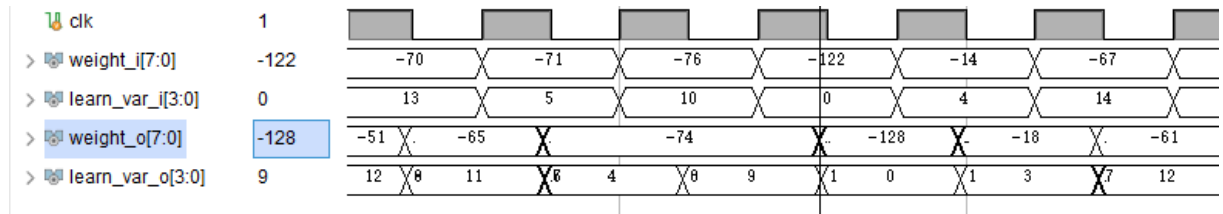


图 5-7 启用 STDP 学习时突触权重更新图

图 5-8 为启用 SDSP 在线学习时权重更新模块的信号波形图。权重根据学习变量的第 2 位值做出相应变化。当学习变量的低 2 位为 2'b01 时，权重值加 1，当学习变量的低 2 位为 2'b10 时，权重值减 1。如果上述两种条件均不满足，则会触发双稳态机制，使大于零的权重值加 1，小于零的权重值减 1。在图中光标处前，学习变量的低 2 位为 2'b10，输入权重位 20，输出权重为 19。在图中光标处后，学习变量的低 2 位为 2'b01，输入权重位 42，输出权重为 43。

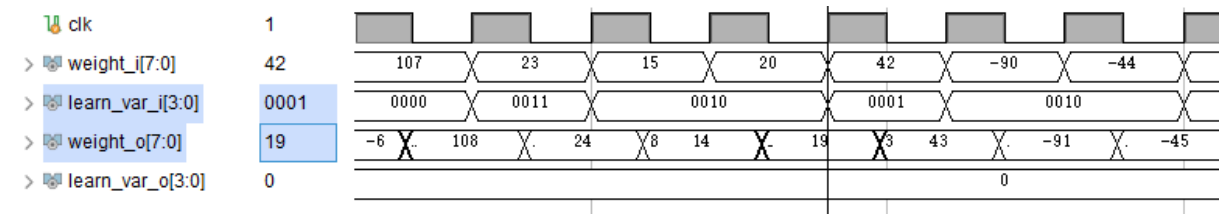


图 5-8 启用 SDSP 学习时突触权重更新图

5.1.3 输入模块仿真测试

本节对第四章设计的输入模块进行仿真测试和分析。使用外界信号发送端向 SNN 加速器发送脉冲数据包，脉冲数据包包括随机的源神经元地址与时间步标识，测试输入模块的数据接收功能和分类功能。在不同的时间步向输入模块发送读输入缓存 FIFO 信号，测试其输出功能。

输入模块的仿真波形图如图 5-9 所示。仿真波形与在第四章介绍的握手协议一致。外部请求信号 AER_rx_req 和输入模块应答信号 AER_rx_ack 交替被置为高电平，并且为了避免亚稳态的出现，输入模块在接收到外部请求信号 AER_rx_req 后，会将其通过两

个寄存器。在输入模块的应答信号 *AER_rx_ack* 拉高的同时，会将对应时间步的 FIFO 写入信号也拉高一个时钟周期（时间步最低位为 0 对应信号为 *FIFO_push_0*，时间步最低位为 1 对应信号为 *FIFO_push_1*），此时输入数据被采样到对应的输入缓存 FIFO 中。当控制模块状态处于 UPDATE II 时，会向输入模块发送读 FIFO 信号，输入模块会根据当前的时间步来判断输出哪个 FIFO 中的数据，当前时间步的最低位为 0 则对应 0 号输入缓存 FIFO，反之则对应 1 号。

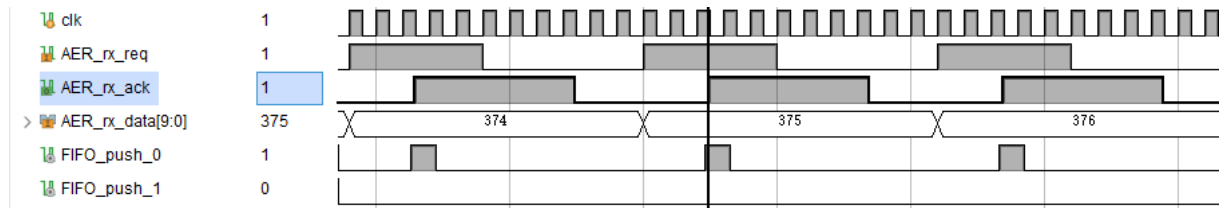


图 5-9 输入模块仿真波形图

5.1.4 控制模块仿真测试

本节对第四章设计的控制模块进行仿真测试和分析。控制模块有四种工作状态，在 IDLE 状态下，各输出信号均为 0；在 UPDATE I 状态下，控制模块会依次输出目标神经元地址；在 UPDATE II 状态下，控制模块会根据输入模块的源神经元地址，依次输出与其具有突触连接的目标神经元地址；在 LEARN 状态下，控制模块会根据当前时间步发放脉冲的目标神经元地址，依次输出与之相连的源-目突触地址。此外，控制模块还会向 LIF 神经元模块发送信号，指定其工作模式。

图 5-10 展示了在 UPDATE I 状态下控制模块主要信号的波形图，其中 *current_state* 为内部信号，其余为输出信号。当控制模块处在 IDLE 状态时，如果当前时刻的输出缓存 FIFO 不为空，则控制模块将会切换到 UPDATE I 状态。

在第 1 个时钟周期，*current_state* 信号从 0 变为 1，表示控制模块已经切换为 UPDATE I 状态，同时目标神经元地址 00 被放到存储模块的读地址线上。在第 2 个时钟周期，*ctrl_step* 信号也变为 1，用于控制 LIF 神经元模块的工作状态。LIF 神经元模块会在该时钟周期内对输入的 00 目标神经元数据进行计算。在第 3 个时钟周期，目标神经元地址 00 会被放在存储模块的写地址线上，同时写神经元使能拉高，将完成计算的目标神经元数据回写到存储模块中。

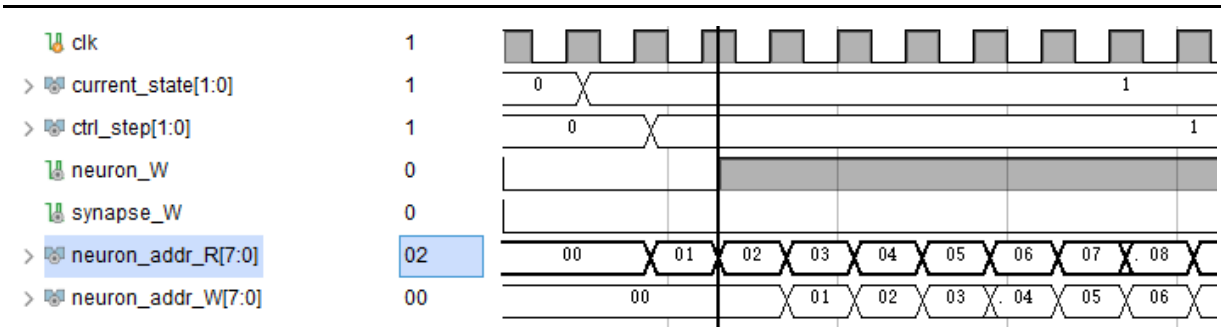


图 5-10 UPDATE I 状态下控制模块仿真波形图

图 5-11 展示了在 UPDATE II 状态下的控制模块主要信号的波形图。控制模块在 UPDATE I 状态下完成所有激活的目标神经元的读操作后（图中为地址 0f 的目标神经元），控制模块的状态会切换为 UPDATE II。

在第 1 个时钟周期, *current_state* 信号从 1 变为 2, 表示控制模块已经切换为 UPDATE II 状态, 同时目标神经元地址 8'h00 和源-目标神经元突触地址 16'h3900 被放到存储模块的读地址线上。由于在本时钟周期 *ctrl_step* 信号仍为 1, 所以 LIF 神经元模块会对输入的 0f 目标神经元数据做 UPDATE I 更新。在第 2 个时钟周期, 8'h00 目标神经元数据和 16'h3900 突触权重数据会被送入 LIF 神经元模块, 此时 *ctrl_step* 信号从 1 变为 2, 控制 LIF 神经元模块进行 UPDATE II 更新。完成 UPDATE I 更新的 8'h0f 目标神经元数据会被回写到存储模块中。在第 3 个时钟周期, 写突触使能拉高, 完成 UPDATE II 更新的 8'h00 目标神经元数据和 16'h3900 突触数据会被回写到存储模块中。

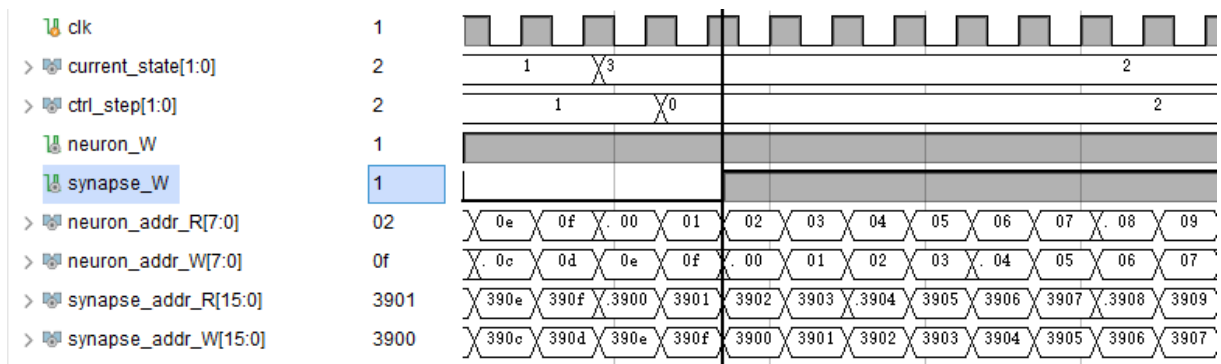


图 5-11 UPDATE II 状态下控制模块仿真波形图

当目标神经元地址等于最大目标神经元地址减 1 时（在本次测试中最大目标神经元地址为 0F），*inputFIFO_pop* 信号会拉高 1 个时钟周期，弹出当前的源神经元地址。新的源神经元地址会在 *inputFIFO_pop* 信号拉高后的第 2 个时钟周期被送入控制模块。如图 5-12 中标记处所示，源神经元地址 8'h4b 被弹出，地址 8'h4c 送入控制模块。

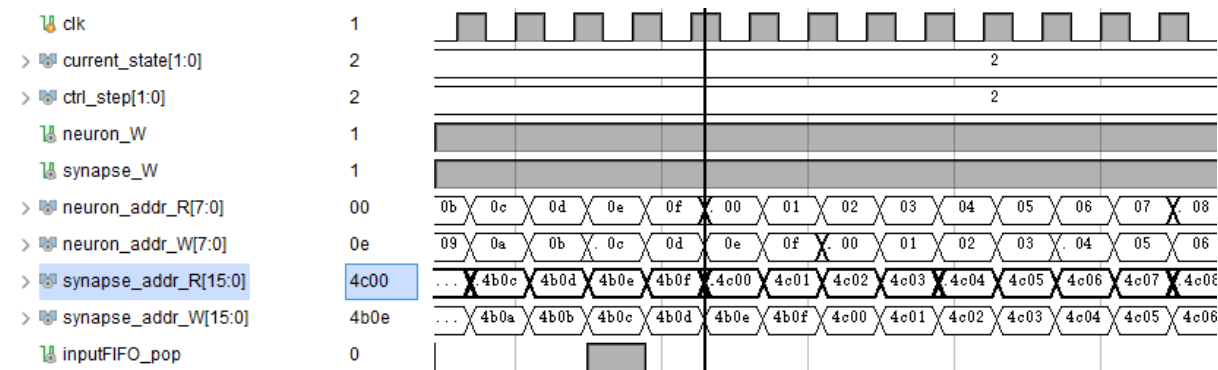


图 5-12 UPDATE II 状态下控制模块完成一轮更新后的波形图

图 5-13 展示了在 LEARN 状态下的控制模块主要信号的波形图。控制模块在 UPDATE II 状态下拉高 *inputFIFO_pop* 后，如果输入缓存 FIFO 变为空，且启用了在线学习功能，则会切换到 LEARN 状态。

在第 1 个时钟周期，*current_state* 信号从 2 变为 3，表示控制模块已经切换为 LEARN 状态，同时从输出缓存 FIFO 中读取发放的目标神经元地址 8'h07，将源-目标神经元突触地址 16'h0007 被放到存储模块的读地址线上。在本时钟周期 *ctrl_step* 信号仍为 2，所以 LIF 神经元模块会对输入的 16'hc90f 目标神经元数据做 UPDATE II 更新。在第 2 个时钟周期，16'h0007 突触权重数据会被送入 LIF 神经元模块，此时 *ctrl_step* 信号从 2 变为 3，控制 LIF 神经元模块进行 LEARN 更新。完成 UPDATE II 更新的 8'h0f 目标神经元数据和 16'hc90f 突触数据会被回写到存储模块中。在第 3 个时钟周期，写突触使能拉高，完成 LEARN 更新的 16'h0007 突触数据会被回写到存储模块中。

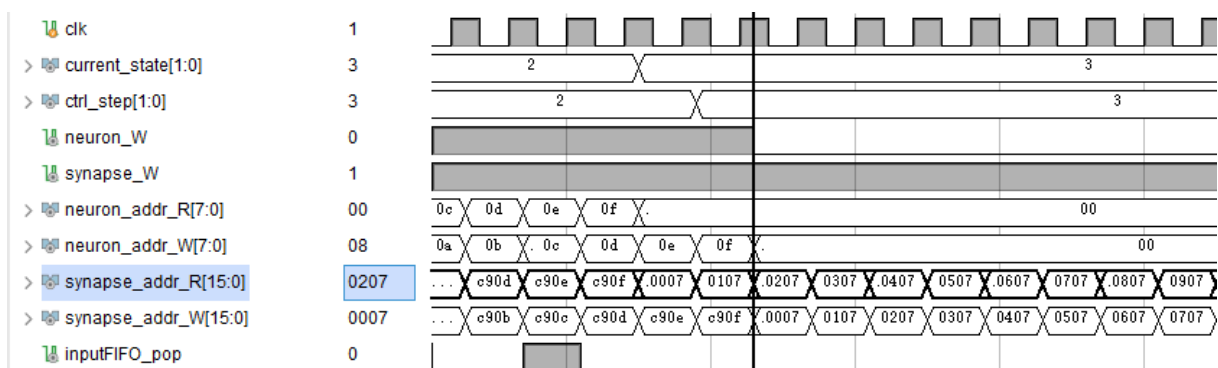


图 5-13 LEARN 状态下控制模块仿真波形图

当源神经元地址等于最大源神经元地址减 1 时（在本次测试中最大源神经元地址为 FF），*outputFIFO_pop* 信号会拉高 1 个时钟周期，弹出当前的目标神经元地址。新的目标神经元地址会在 *outputFIFO_pop* 信号拉高后的第 2 个时钟周期被送入控制模块。如

图 5-14 中标记处所示，目标神经元地址 8'h07 被弹出，地址 8'h0d 送入控制模块。

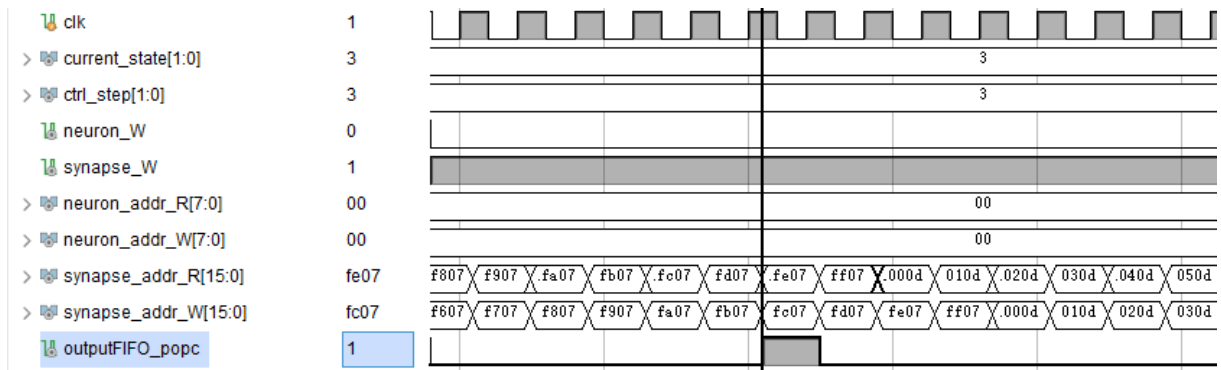


图 5-14 LEARN 状态下控制模块完成一轮更新后的波形图

5.1.5 输出模块仿真测试

本节对第四章设计的输出模块进行仿真测试和分析。当 SNN 加速器中的 LIF 神经元模块达到发放条件产生脉冲后，会先将达到发放条件的目标神经元地址放入输出缓存 FIFO 中，如果没有启动在线学习，输出模块则会读取输出缓存 FIFO 中的数据，利用握手协议发送生成的脉冲数据包。

如图 5-15 所示，当目标神经元 0d 发放时，输出缓存 FIFO 不为空，握手请求信号会拉高，输出模块开始通过握手协议传递数据，可以看到脉冲数据包传递的数据为 20d，其中首个数字 2 代表二进制的 10，1 说明了该脉冲数据包有效，0 则表示该脉冲发放的时间。握手请求信号结束时，*outputFIFO_pop* 信号拉高一个时钟周期，用于弹出输出缓存 FIFO 中已经发送过的数据。

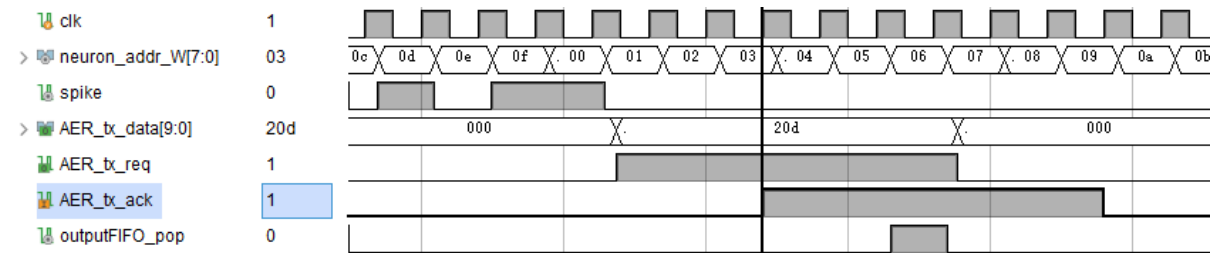


图 5-15 输出模块仿真波形图

5.2 整体仿真测试

本节对 SNN 加速器整体进行仿真测试和分析。首先，设置神经元初始膜电位为 0，并随机生成突触权重，通过.coe 文件载入到 SNN 加速器的存储模块中，如图 5-16 所示。接着设置神经元的常数参数，如图 5-17 所示。然后通过外部的 AER 输出模块，向 SNN 加速器发送脉冲数据包，通过观察波形，分析 SNN 加速器功能的正确性。

<pre>memory_initialization_radix = 2; memory_initialization_vector = 000000000000000000000000, 000000000000000000000000, 000000000000000000000000, 000000000000000000000000, 000000000000000000000000,</pre>	<pre>memory_initialization_radix = 2; memory_initialization_vector = 000000000010, 000000000100, 000000000010, 000000000011,</pre>
a)	b)

图 5-16 载入的数据。(a) 神经元数据，(b) 突触数据

```
memory_initialization_radix = 16;
memory_initialization_vector =
D8301005,
00010108,
000005DC,
00000FFF;
```

图 5-17 神经元常数参数设置

神经元的常数参数保存在一个位宽 32bit，深度为 4 的 RAM 中。文件中数据的第一行 D8301005 表示神经元的学习算法参数，第二行 00010108 仅低 20 位有效，表示神经元参数，第三行 000005DC 仅低 16 位有效，表示每个时间步持续多少个时钟周期，第四行 00000FFF 仅低 18 位有效，表示 SNN 加速器的系统设置，包括是否启用在线学习，在线学习模式，启用的目标神经元数量和源神经元数量，具体分布如表 5-2 所示。

表 5-2 SNN 加速器系统设置

位置	信号名称	位宽 (bit)	描述
7-0	<i>src_number</i>	8	启用源神经元数量
15-8	<i>tar_number</i>	8	启用目标神经元数量
16	<i>learn_mode</i>	1	在线学习模式
17	<i>enable_learn</i>	1	是否启用在线学习

关闭在线学习功能,SNN加速器在第一个时间步的整体仿真波形图如图 5-18 所示。光标位置的脉冲发放标志 *spike* 值为 1,说明有目标神经元发放脉冲。图 5-19 到图 5-22 为将整体仿真波形图局部放大后的图片。

图 5-19 所示为 SNN 加速器刚开始工作时,此时输入模块接收到了脉冲数据包,当前时间步的输入缓存 FIFO 非空,控制模块状态从 IDLE 转换为 UPDATE I。

图 5-20 所示为 SNN 加速器已经完成了 UPDATE I 状态的更新,转为 UPDATE II 状态时的波形图。

图 5-21 所示为 SNN 加速器在 UPDATE II 状态下已经完成了第 1 个源神经元脉冲输入的更新,开始更新第 2 个源神经元脉冲输入时的波形图。在第 1 个光标处可以看到目标神经元地址为 8'h00,权重地址为 16'h3900,其值在下一个时钟周期被读出,神经元膜电位为 16'h0000,突触权重为 8'h01,在第 2 个光标处输出膜电位计算结果 16'h0001,计算结果正确。

图 5-22 所示为 SNN 加速器中首次有目标神经元膜电位到达阈值(阈值为 16'd264),发放脉冲时的波形图。在有目标神经元发放后经过 3 个时钟周期,输出模块的握手请求信号拉高,开始向外界发送脉冲数据包。

开启 STDP 在线学习功能,根据设置 *xtar* 的值为 5, SNN 加速器在第一个时间步的整体仿真波形图如图 5-23 所示。当控制模块状态为 IDLE、UPDATE I 和 UPDATE II 时,是否开启在线学习功能的仿真波形基本相同,对 LEARN 状态的仿真波形进行分析,如图 5-24 所示。

在图 5-24 中,SNN 加速器完成了 UPDATE II 状态的更新,因为有目标神经元发放,且启用了在线学习功能,所以控制模块的状态转为 LEARN。由图 5-22 可知,膜电位首先到达阈值并发放的目标神经元地址是 8'h07,所以首先对其权重进行更新的目标神经元地址也是 8'h07。在第一个光标处可以看到输入权重地址 32'h0007,其值在下一个时钟周期被读出为 12'h003,在第二个光标处输出计算结果 12'h0fe,计算结果正确。

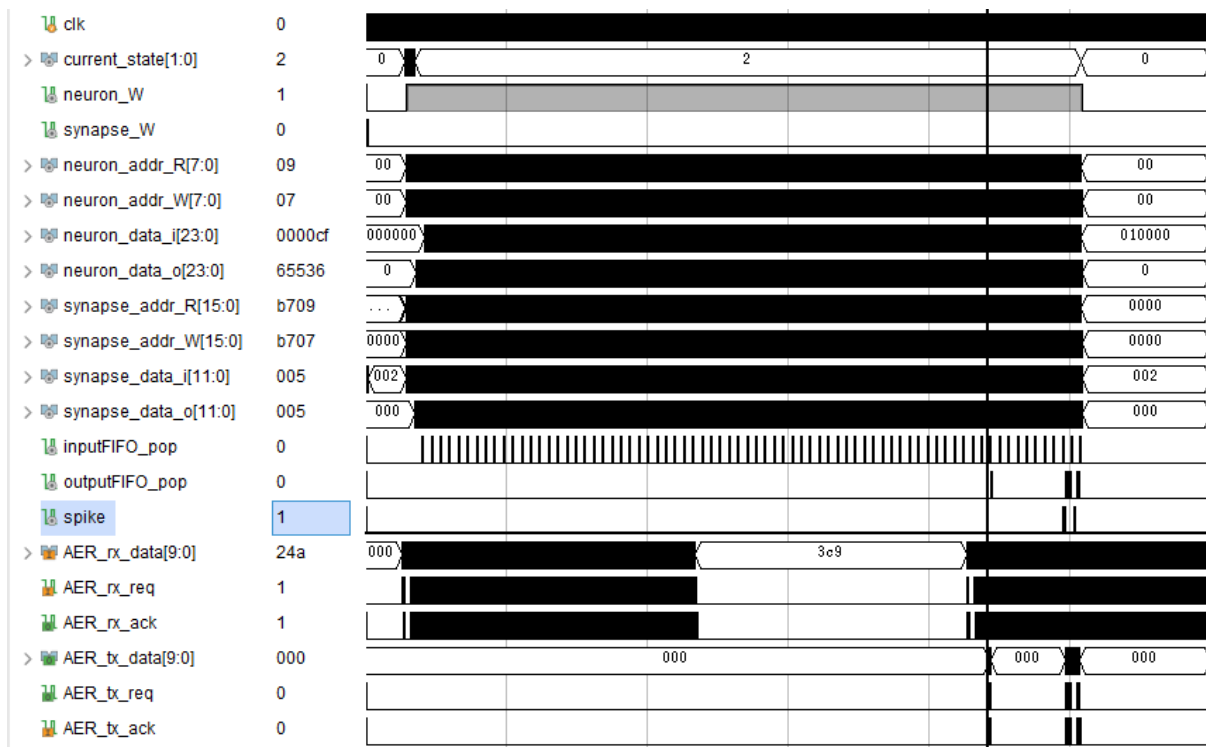


图 5-18 关闭在线学习的整体波形仿真图

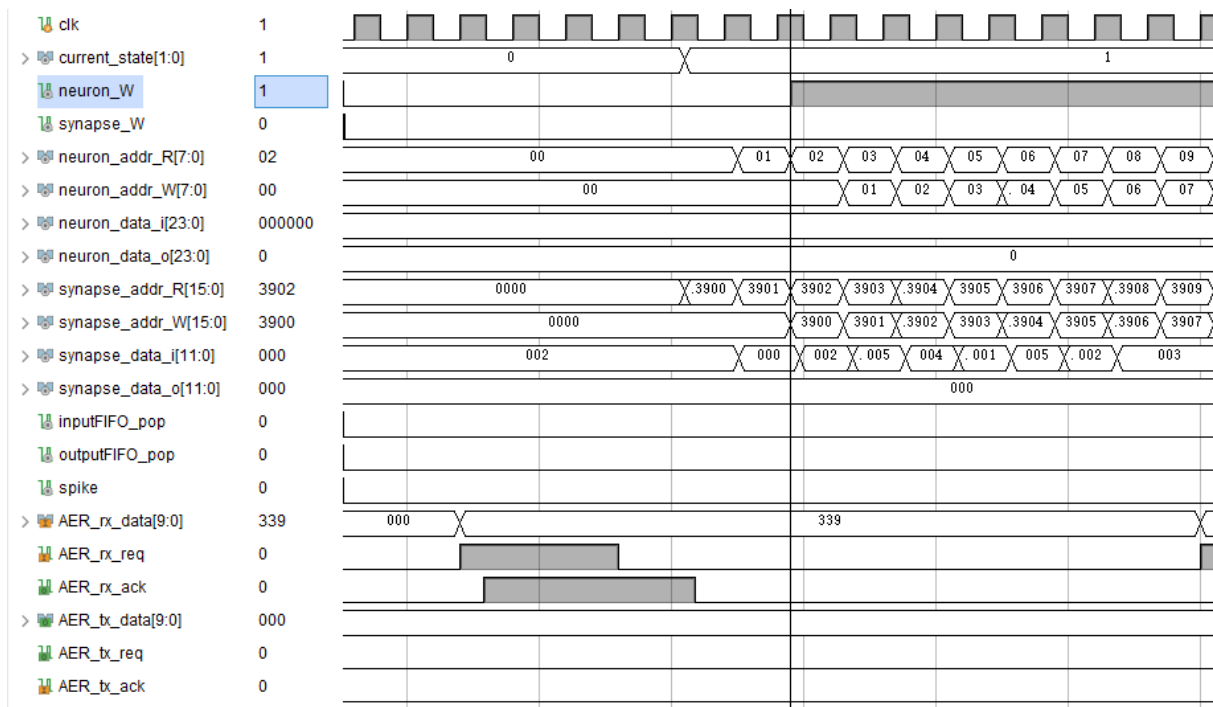


图 5-19 关闭在线学习的局部放大图 I

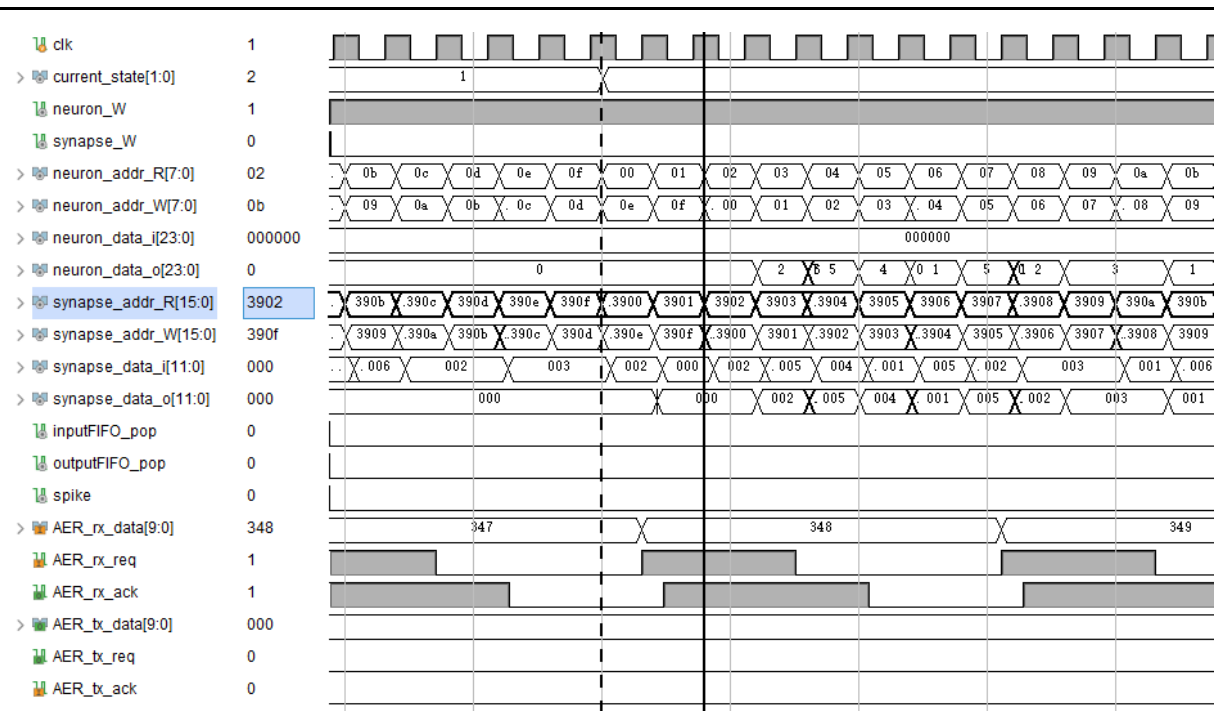


图 5-20 关闭在线学习的局部放大图 II

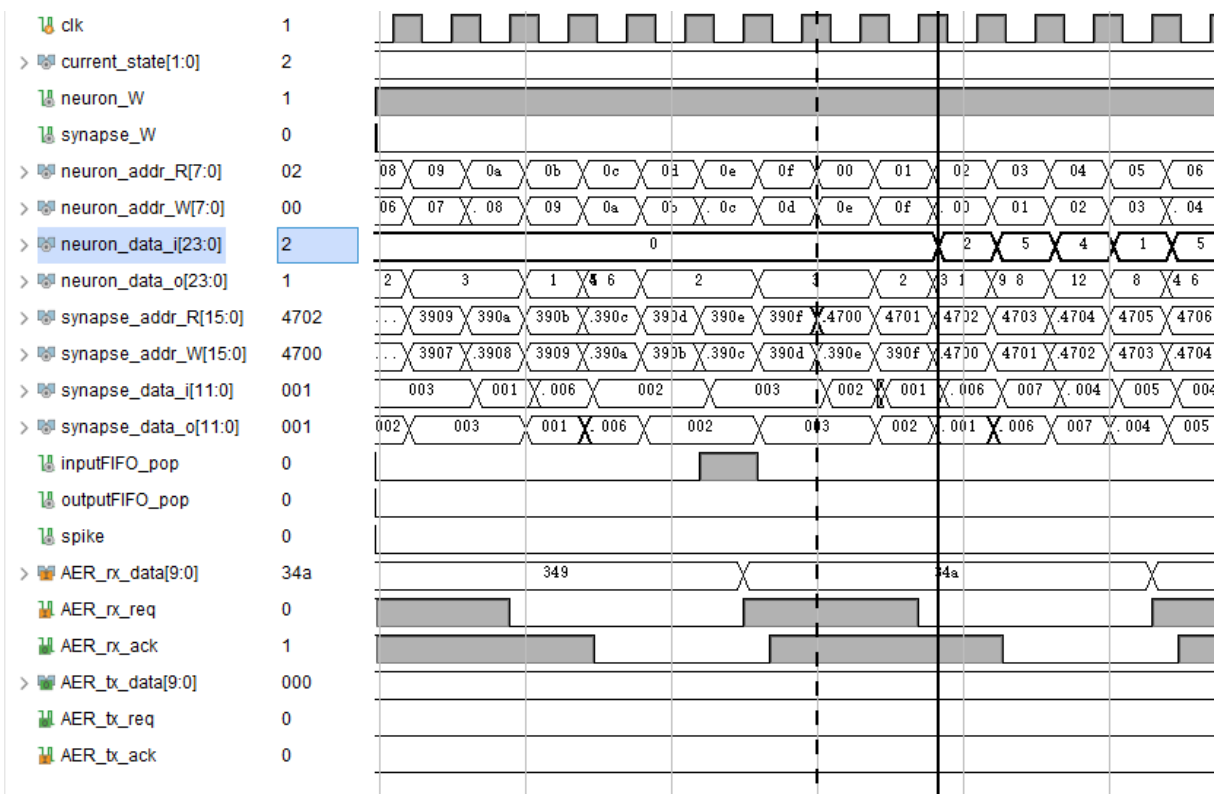


图 5-21 关闭在线学习的局部放大图 III

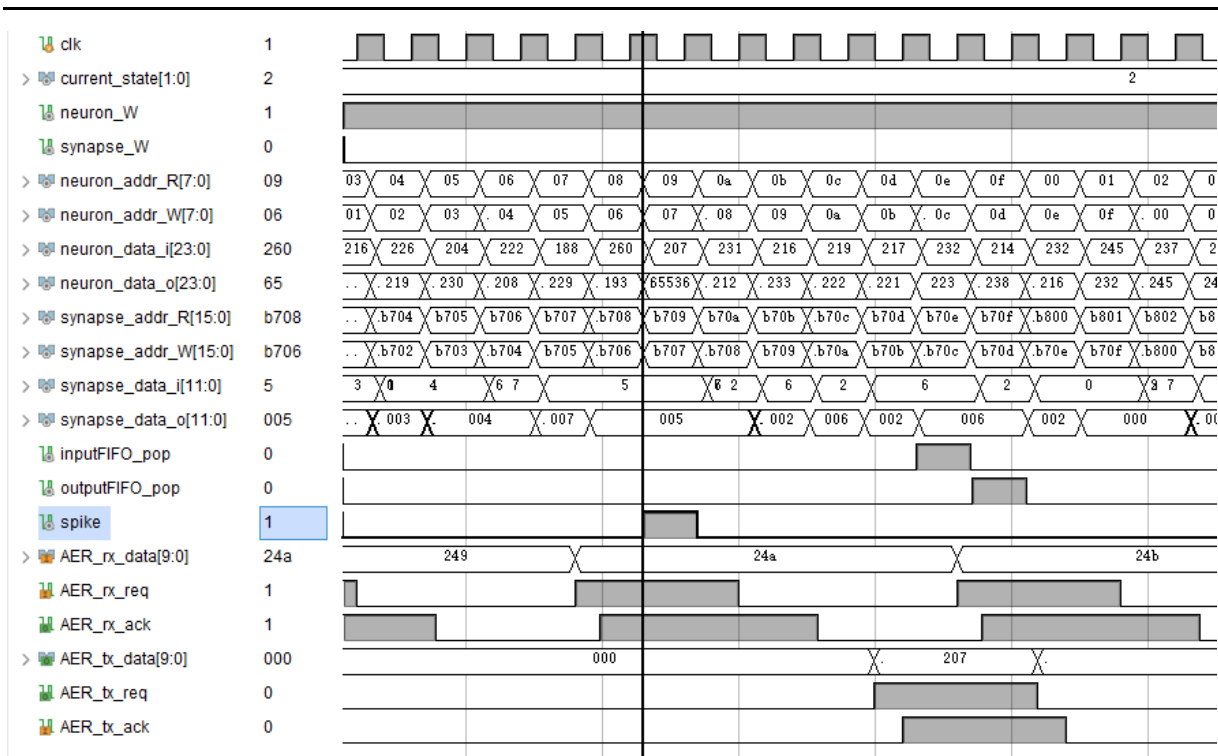


图 5-22 关闭在线学习的局部放大图 IV

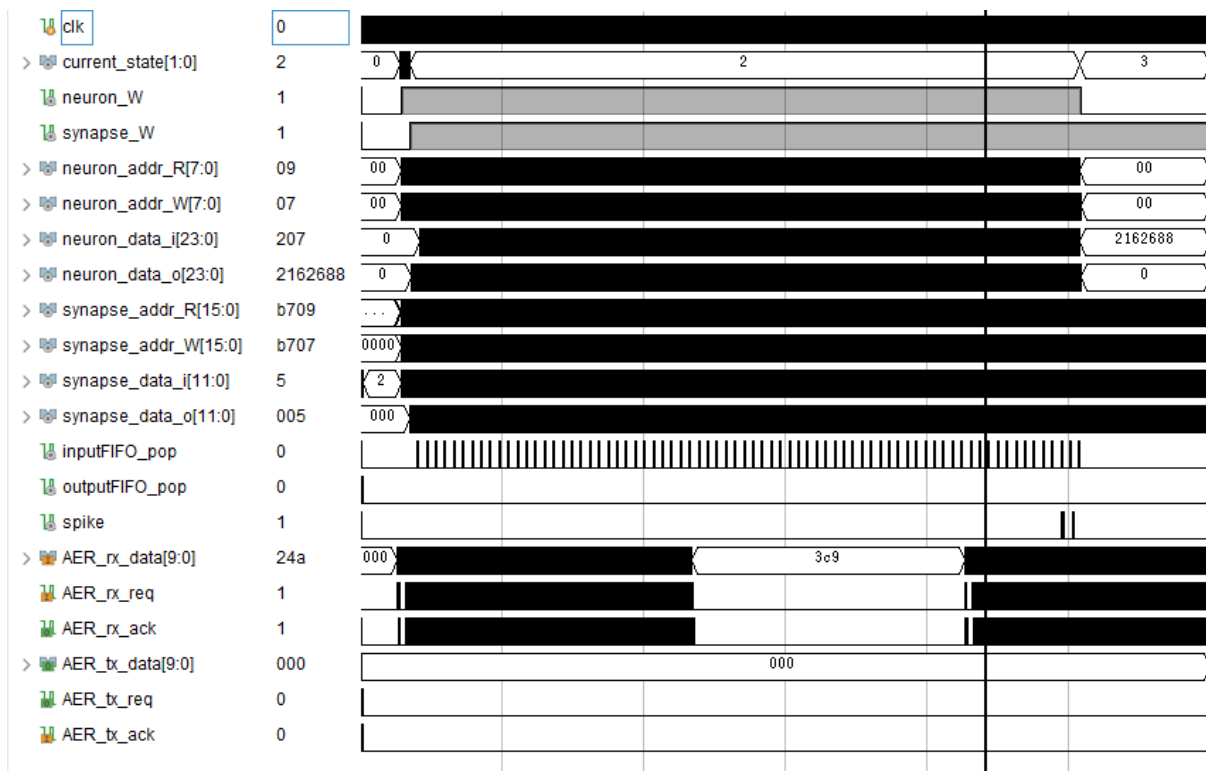


图 5-23 开启在线学习的整体波形仿真图

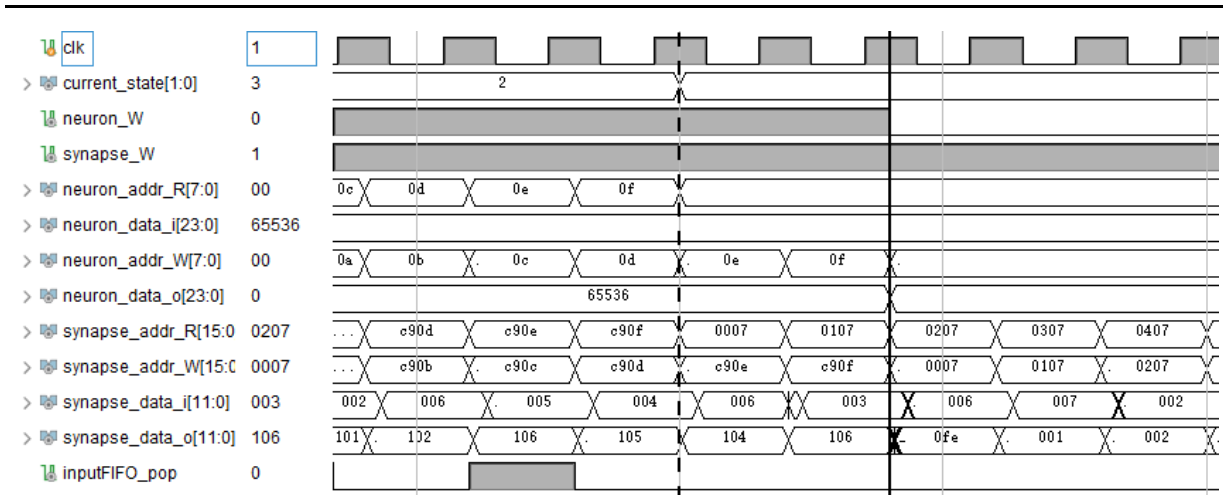


图 5-24 开启在线学习的局部放大图

5.3 网络仿真测试

5.3.1 网络配置

待训练的网络采用前馈型脉冲神经网络，网络的第 1 层为输入层，第 2 层为竞争层，第 1 层与第 2 层的连接方式为全连接，神经元数目均为 16x16 个，与 SNN 加速器所支持的最大神经元数量相同。输入层对于上文的源神经元，竞争层对于上文的目标神经元。设置 STDP 学习参数 $xtar$ 的值为 4'd8，阈值电位设置为 16'h4000，由于规定低七位为小数，所以阈值电位对应为 128，神经元不应期设置为 1，表示在一个时间步内发放过的神经元不会再进行发放。

神经元之间通常采用侧抑制的方法来实现竞争学习，复杂的侧抑制方法与时间、空间均有关系，仿真与实现较为复杂且资源消耗大。在本设计的竞争层中，采用了简化的侧抑制方法：在一个时间步内，如果有神经元到达阈值电位，则所有神经元都会复位到静息电位，但是仅会对到达阈值电位的神经元进行突触更新。

5.3.2 网络数据的输入

网络选用 MNIST 手写数据集进行训练。MNIST 数据集中的图片大小为 28x28，首先对其进行最大池化，使图片大小变为 16x16，以满足 SNN 加速器的输入要求。图 5-25 分别展示了池化前与池化后的手写数字 5，为了方便观察对原始灰度图像进行了处理，新值等于 1 减去原始灰度图像的值。接着，对图片进行泊松脉冲编码，泊松脉冲编码的

方法具体如公式（5-1）所示，在每个时间步内，每个像素点发放脉冲的概率与该像素值成正比。对池化后的手写数字 5 进行 10 个时间步的脉冲编码如图 5-26 所示。记录下编码后每个时间步发放脉冲的像素点的编号，这就是输入到加速器中的源神经元地址。最后，将源神经元地址和发放脉冲的时间步整合起来生成脉冲数据包，发送到待训练的网络。

$$P(X = k) = \frac{\lambda^k}{k!} e^{-\lambda} \quad (5-1)$$



图 5-25 池化前后的手写数字 5

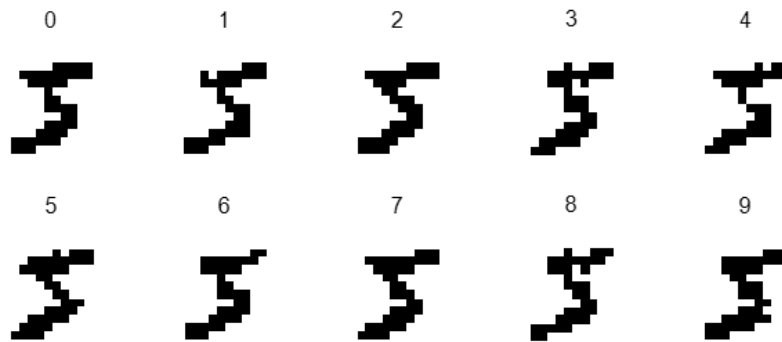


图 5-26 脉冲编码后的手写数字 5

5.3.3 网络训练

我们将 MNIST 数据集中的图片依次输入到网络中，网络按照如下方式进行训练：

- （1）初始化所有神经元的膜电位，使其等于静息电位。随机生成取值范围为（0，1）的突触权重，设置初始权值为正防止神经元膜电位难以到达阈值。
- （2）将脉冲编码后的手写数字图片发送给脉冲神经网络，每个图片的呈现时间为 250 个时间步。

在训练开始时，由于初始权重是随机分布的，所以每个神经元的膜电位随机增加，神经元的发放没有规律。但随着训练的进行，发放脉冲的神经元会根据 STDP 学习算法来对自身突触权重进行改变同时由于侧抑制，没有发放脉冲的神经元也会失去所有已经累积的膜电位值，并且不能改变自身突触权重。进而导致每个神经元会对特定的输入较为敏感。当这些输入出现时，神经元会以较高的频率发放脉冲。而当其他输入出现时，神经元几乎不发放脉冲。

训练结束后，输入层到竞争层的连接权重如图 5-27 所示，对于竞争层的每个神经元，其 256 个突触权重被重新排列为 16×16 的矩阵。竞争层共有 16×16 个神经元，所以图 5-27 的大小为 $(16 \times 16) \times (16 \times 16) = 256 \times 256$ 。

图 5-27 直观的展示了每个神经元对应的 16×16 权重矩阵形成了 0-9 中的某个数字，说明该神经元学习到了该类数字的特征。图 5-28 展示了训练前和训练后的权重分布，训练前的权重为随机分布，且根据设定均大于零。训练后的权重集中在最大值 1 和最小值 -1 附近。

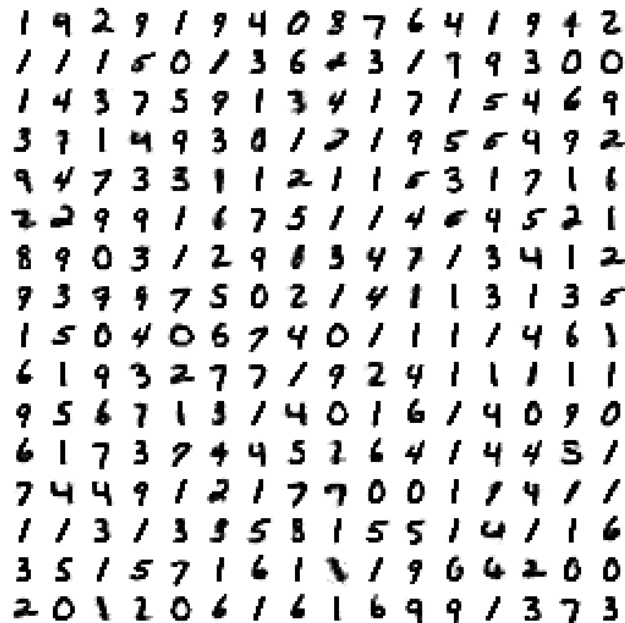


图 5-27 训练后的连接权重图

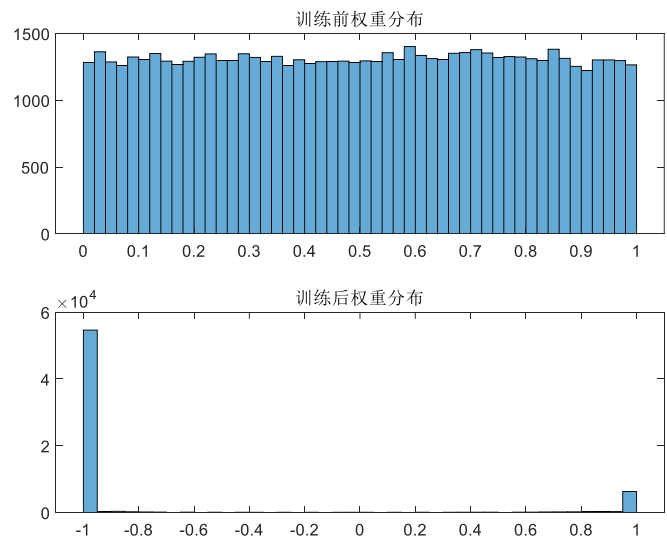


图 5-28 训练前后的权重分布

5.4 资源开销与功耗分析

我们选用 xc7k70tfbv676-1 型号的 FPGA 对 SNN 加速器进行综合（Symthesis）与实现（Implementation）。图 5-29 所示为加速器所消耗的资源，可以看到除了 BRAM 之外，加速器的资源占用并不高。

Resource	Utilization	Available	Utilization...
LUT	428	41000	1.04
FF	455	82000	0.55
BRAM	24.50	135	18.15
IO	26	300	8.67
BUFG	1	32	3.13

图 5-29 SNN 加速器的资源开销

实现后的时序分析如图 5-30 所示，在 100MHz 条件下，保持时间和建立时间均满足要求。SNN 加速器的输入时钟频率为 100MHz，电压为 1.8V，功耗分析如图 5-31 所示，加速器总功耗为 0.116W，其中静态功耗占 71%。

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.955 ns	Worst Hold Slack (WHS): 0.056 ns	Worst Pulse Width Slack (WPWS): 4.600 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1641	Total Number of Endpoints: 1641	Total Number of Endpoints: 510
All user specified timing constraints are met.		

图 5-30 SNN 加速器的时序分析

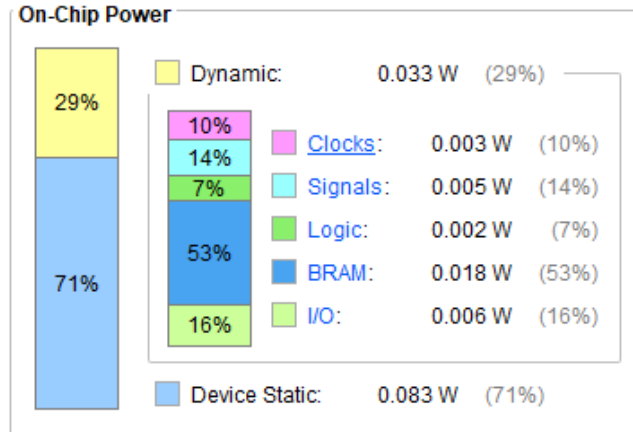


图 5-31 SNN 加速器的功耗分析

表 5-3 是本设计与典型的 SNN 加速器进行对比。本文设计的 SNN 加速器支持 STDP 和 SDSP 两种在线学习机制，同时具有较高的突触操作速率，但是由于本文使用的是 FPGA 实现，在功耗和面积方面难以直接进行对比。

表 5-3 SNN 加速器参数对比

芯片名称	实现方式	在线学习机制	神经元数目	突触数目	突触操作速率
SpiNNaker ^[26]	0.13 μ m	STDP	1000	-	16.56MSOP/s
TrueNorth ^[4]	28nm	No	256	64K	-
Loihi ^[27]	14nm	STDP	1024	1M-114K	100MSOP/s
ODIN ^[5]	28nm	SDSP	256	64K	37.5MSOP/s
本设计	FPGA	STDP/SDSP	256	64K	100MSOP/s

5.5 本章小结

本章首先测试了第三章设计的 LIF 神经元模块和权重更新模块的功能，并展示了神经元的动态更新过程。接着对第四章中设计的各个模块进行了测试：输入模块与输出模块对脉冲数据包的接收与发送，控制模块对加速器内各个控制信号的分配，神经元数据与突触数据的读取与写入，时间步的生成等。测试结果说明所有模块的功能正确，满足设计需求。然后使用 MNIST 数据集与 STDP 学习算法对网络进行训练，训练后的权重反映出了手写数字的特征，表明所设计的 SNN 加速器可以正确运行。最后分析了 VIVADO implementation 后 SNN 加速器的资源开销和功耗，可以看到加速器的资源开销较小，且动态功耗仅有 33mW，说明设计的加速器具有低功耗的特性。

结论

1. 论文工作总结

本研究旨在基于 FPGA 设计一种高效的脉冲神经网络加速器，以充分发挥脉冲神经网络在神经计算中的潜力。首先，在论文的第一章中，我们详细介绍了脉冲神经网络的优势。与传统的人工神经网络相比，SNN 具有稀疏编码、事件驱动计算和能量效率等优势，适用于各种实时和低功耗的应用场景。

第二章系统地介绍了脉冲神经网络的原理。我们从神经元模型、神经网络结构到脉冲编码方式和运算规则，对 SNN 的各个方面进行了详细阐述。这为后续的硬件设计提供了理论基础和指导，使我们能够更好地理解 SNN 的工作机制并将其转化为硬件实现。

在第三章中，我们重点设计了基于 LIF 神经元模型的实现。通过详细分析 LIF 神经元的数学模型和运算过程，我们设计了相应的硬件电路，并对其进行了优化以满足实时性和资源利用率的要求。LIF 神经元作为 SNN 的基本单元，在加速器中起着至关重要的作用，其设计质量直接影响了整体系统的性能。

第四章提出了脉冲神经网络加速器的整体架构。我们设计了控制模块、存储模块和输入输出模块等关键组件，并详细描述了它们之间的协同工作机制。特别地，我们采用了分布式存储和并行计算的设计思想，以充分发挥 FPGA 硬件的并行计算能力，提高加速器的运算效率和吞吐量。

最后，在第五章中，我们使用 FPGA 对设计的 SNN 加速器进行综合实现。设计了仿真测试模型，对 SNN 加速器的参数进行配置，分别验证了各个模块功能与时序的正确性。将 SNN 加速器配置成一个单层前馈型脉冲神经网络，使用 MNIST 数据集和 STDP 学习规则对网络进行训练，结果表明突触权重正确地学习到了数字的特征。测试结束后对加速器的资源开销和功耗等进行统计，展示了加速器具有低功耗和低资源开销的特性。

我们设计的加速器在性能和能效方面都具有优势，能够有效地加速 SNN 的计算过程，为边缘设备上的神经计算应用提供了新的解决方案和可能性。

2. 工作展望

基于我们在论文中所设计的脉冲神经网络加速器，未来有许多方向可以继续探索和拓展：

（1）支持更多类型的数字神经元模型。在本设计中，SNN 加速器仅支持对以 LIF 神经元为基础的脉冲神经网络进行计算。在 SNN 领域内，IZH 神经元、H-H 神经元等模型也是研究的热点。通过支持更多类型的神经元模型，可以增加 SNN 加速器的适用性。

（2）设计路由模块，构建更大规模片上脉冲神经网络。通过将 SNN 加速器作为 PE 节点，由路由模块将脉冲数据包发送给 SNN 加速器，可以实现对更大规模的脉冲神经网络进行计算。对于如何设计网络的拓扑结构、数据的交换机制和路由算法，还需要进一步的学习与研究。

（3）使用 ASIC 实现，进一步提高速度、降低功耗。基于 FPGA 可以快速完成 SNN 加速器的设计和验证，当完善加速器的功能之后，可以对其进行 ASIC 实现。对于在 FPGA 实现中占用功耗较大的 BRAM，可以选用 SRAM 来进行优化，从而进一步提高速度、降低功耗。

参考文献

- [1] 张铁林,徐波.脉冲神经网络研究现状及展望[J].计算机学报,2021,44(09):1767-1785.
- [2] Sandin F, Khan A I, Dyer A G, et al. Concept Learning in Neuromorphic Vision Systems: What Can We Learn from Insects[J]. Journal of Software Engineering and Applications 7 (2014): 387-395.
- [3] Li S, Zhang Z, Mao R, et al. A Fast and Energy-Efficient SNN Processor With Adaptive Clock/Event-Driven Computation Scheme and Online Learning[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2021, PP(99):1-10. DOI:10.1109/TCSI.2021.3052885.
- [4] Akopyan F, Sawada J, Cassidy A S, et al. TrueNorth: Design and Tool Flow of a 65 mW 1 Million Neuron Programmable Neurosynaptic Chip[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2015, 34(10): 1537-1557.
- [5] Frenkel C, Lefebvre M, Legat J D, et al. A 0.086-mm² 12.7-pJ/SOP 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS[J]. IEEE Transactions on Biomedical Circuits and Systems, 2018, 13(1): 145-158.
- [6] Pei J, Deng L, Song S, et al. Towards artificial general intelligence with hybrid Tianjic chip architecture[J]. Nature, 2019, 572(7767): 106-111.
- [7] 程龙, 刘洋. 脉冲神经网络: 模型、学习算法与应用[J]. 控制与决策, 2018, v.33(05):158-172.
- [8] Gerstner W, Kistler W M, Naud R, et al. Neuronal dynamics: From single neurons to networks and models of cognition[M]. Cambridge University Press, 2014(1): 2-25.
- [9] Ghosh-Dastidar S, Adeli H. Spiking neural networks[J]. International journal of neural systems, 2009, 19(04): 295-308.
- [10] Forrest M D. Can the thermodynamic Hodgkin-Huxley model of voltage-dependent conductance extrapolate for temperature?[J]. Computation, 2014, 2(2): 47-60.
- [11] Izhikevich E M. Which model to use for cortical spiking neurons?[J]. IEEE Transactions on Neural Networks, 2004, 15(5): 1063-1070.
- [12] Abbott L F. Lapicque's introduction of the integrate-and-fire model neuron (1907)[J]. Brain Research Bulletin, 1999, 50(5-6): 303-304.

-
- [13] Heidarpur M, Ahmadi A, Ahmadi M, et al. CORDIC-SNN: On-FPGA STDP learning with izhikevich neurons[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2019, 66(7): 2651-2661.
- [14] Cao Y, Chen Y, Khosla D. Spiking deep convolutional neural networks for energy-efficient object recognition[J]. International Journal of Computer Vision, 2015, 113(1): 54-66.
- [15] 胡一凡,李国齐,吴郁杰,等. 脉冲神经网络研究进展综述[J]. 控制与决策, 2021, 36(1): 1-26.
- [16] Bohte S M, Kok J N, La Poutre H. Error-backpropagation in temporally encoded networks of spiking neurons[J]. Neurocomputing, 2002, 48(1-4): 17-37.
- [17] Gütig R, Sompolinsky H. The tempotron: a neuron that learns spike timing-based decisions[J]. Nature Neuroscience, 2006, 9(3): 420-428.
- [18] Gupta A, Long L N. Hebbian learning with winner take all for spiking neural networks[C]. 2009 International Joint Conference on Neural Networks. IEEE, 2009: 1054-1060.
- [19] Markram H, Gerstner W, Sjöström P J. Spike-timing-dependent plasticity: a comprehensive overview[J]. Frontiers in Synaptic Neuroscience, 2012, 4: 2.
- [20] Diehl P U, Cook M. Unsupervised learning of digit recognition using spike-timing-dependent plasticity[J]. Frontiers in Computational Neuroscience, 2015, 9:99
- [21] Brea J, Senn W, Pfister J P. Matching recall and storage in sequence learning with spiking neural networks[J]. Journal of Neuroscience, 2013, 33(23): 9565-9575.
- [22] Paugam-Moisy H, Martinez R, Bengio S. Delay learning and polychronization for reservoir computing[J]. Neurocomputing, 2008, 71(7-9): 1143-1158.
- [23] Zhang J, Wang R, Pei X, et al. A fast spiking neural network accelerator based on BP-STDP algorithm and weighted neuron model[J]. IEEE Transactions on Circuits and Systems II: Express Briefs, 2021, 69(4): 2271-2275.
- [24] Brader J M, Senn W, Fusi S. Learning real-world stimuli in a neural network with spike-driven synaptic dynamics[J]. Neural computation, 2007, 19(11): 2881-2912.
- [25] Frenkel C, Indiveri G, Legat J D, et al. A fully-synthesized 20-gate digital spike-based synapse with embedded online learning[C]. 2017 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2017: 1-4
- [26] Painkras E, Plana L A, Garside J, et al. SpiNNaker: A 1-W 18-Core System-on-Chip for

-
- Massively-Parallel Neural Network Simulation[J]. IEEE Journal of Solid-State Circuits, 2013, 48(8): 1943-1953.
- [27] Davies M E, Srinivasa N, Lin T, et al. Loihi: A Neuromorphic Manycore Processor with On-Chip Learning[J]. IEEE Micro, 2018, 38(1): 82-99.

致谢

转眼间，大学生活接近尾声。在华工度过的这四年，我经历了许多的挑战，学到了宝贵的知识，也在成长的路上不断前行。这段时光让我从一个懵懂无知的少年蜕变为有责任担当的青年，为我的大学生活画上了完美的句号。在此，我想衷心感谢所有给予我帮助和支持的人。

首先，感谢我的导师姚恩义教授。姚教授治学严谨，知识渊博。在我完成本科毕业设计的过程中，姚教授总是耐心地解答我的问题。他指导我克服了一个个技术难题，让我能够顺利完成毕业设计。在我对于未来学习生活的选择上，姚教授从自己的经历出发，给出了宝贵的建议，让我对未来不再迷惘。

其次，感谢班主任李娜老师和辅导员于义永老师。在我本科学习期间，他们一直给予我关心和帮助，为我提供了良好的学习环境和学习资源，让我能够全身心投入到学业中。

此外，感谢我的父母。是他们一直以来的支持和鼓励，是我求学道路上最坚实的后盾。他们的爱和付出是我前进的动力，让我能够坚持不懈地追求自己的梦想。

最后，感谢所有同学与朋友。是他们给予了我无私的帮助与支持，让我感受到了团队的力量与友谊的温暖。